RISC-V IOPMP Architecture Specification

RISC-V IOPMP Task Group

Version 0.8.2, February, 2026: This document is in development. Assume everything can change. See http://riscv.org/spec-state for details.

Table of Contents

Preamble

This document is in the Development state

Assume everything can change. This draft specification will change before being accepted as standard, so implementations made to this draft specification will likely not conform to the future standard.

Copyright and license information

Contributors

This RISC-V specification has been contributed to directly or indirectly by (in alphabetical order):

Allen Baum,
Alvin Che-Chia Chang,
Andrew Dellow,
Channing Tang,
Dean Liberty,
Erhu Feng,
Geoffrey Thorpe,
Joe Xie,
Luis Cunha,
Nicholas Wood,
Nick Kossifidis,
Oscar Jupp,
Paul Shan-Chyun Ku,
Perrine Peresse,
Siqi Zhao,
TY Ting-Yu Shyu,
Vincenzo Izzo,

Chapter 1. Introduction

This document outlines a mechanism to enhance platform security. In a platform, bus requesters (initiators) can access target devices, similar to a RISC-V hart. Introducing I/O agents like DMA (Direct Memory Access) units improves performance, but also exposes the system to vulnerabilities such as DMA attacks. In the RISC-V ecosystem, the PMP/ePMP [1] provides a standard protection scheme for accesses from a RISC-V hart to the physical address space. However, there is no equivalent standard for safeguarding non-RISC-V CPU requesters. The Physical Memory Protection Unit for Input/Output Devices (IOPMP) addresses this by controlling accesses issued by bus requesters.

IOPMP is a hardware component within a bus fabric to address situations where pure-software solutions are insufficient. For a RISC-V-based platform, a software solution typically involves a security monitor, a program running in M-mode that handles security-related requests. When a request for a DMA transfer is made, the security monitor checks if it meets all security rules and decides if it is legal. Only legal requests are executed. However, this check introduces significant latency overhead, which may violate real-time constraints, especially for high-throughput requests like those from a GPU (Graphics Processing Unit). A hardware component that can check accesses in real-time is a more practical solution. This document focuses on IOPMP. Figure 1 conceptually illustrates the integration of IOPMP into a system as an example.

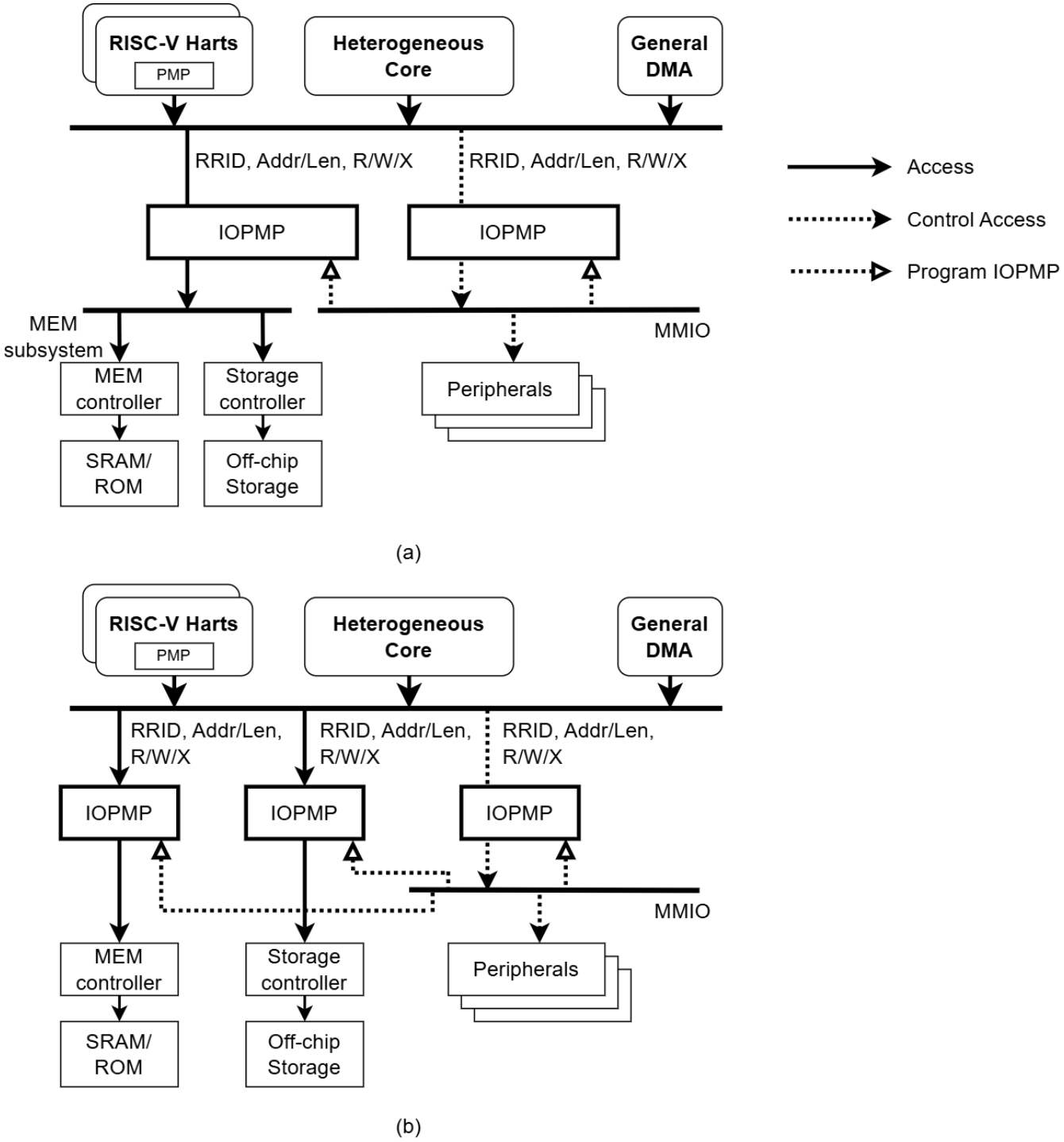

Figure 1. example integrations of IOPMP(s) in a system. Figure (a) shows a system deploying two IOPMPs. The IOPMPs take three inputs: RRID, the transaction type (read/write/instruction fetch), and the request range (address/len). Figure (b) shows a system deploying multiple IOPMPs. In this deployment, IOPMPs are at each resource, final destination.

Both IOMMU and IOPMP have their own useful applications, and they can coexist and collaborate within a bus matrix. The Input-Output Memory Management Unit (IOMMU) [2] is another hardware component in a bus matrix that translates a device's virtual address to a physical address in the bus matrix using one or more stages of page tables. It can operate in both trusted and untrusted environments and is typically used by a hypervisor or an OS. It offers excellent flexibility and reduces external memory fragmentation. However, it may not be ideal for security software running in M-mode because it requires relatively large memory and complex software compared to a typical security monitor. Additionally, the tables are usually stored in DRAM, which is outside the chip and may need external signal protection. For resource-constrained embedded systems, the silicon area and memory footprint required for an IOMMU may be prohibitive. In contrast, the IOPMP consumes much less memory and stores rules inside the chip. IOPMP can be used by the security monitor to check access from devices controlled by other software, while

IOMMU can be used by software with richer resources.

Chapter 2. Terminology and Concepts

This document refers to the term “secure monitor” as the software responsible for managing security-related tasks, including the programming of IOPMPs. The secure monitor is not restricted to operating on a single CPU or hart; instead, it may be distributed across multiple CPUs.

Table 1. Glossary and Acronyms

Term	Description
AMO	Atomic memory operation.
DC	Don’t care.
DMA	Direct memory access.
DRAM	Dynamic random-access memory.
ID	Identifier.
IMP	Implementation-dependent.
IMSIC	Incoming Message-Signaled Interrupt Controller, defined in the RISC-V advanced interrupt architecture (AIA) [3] specification.
IOMMU	Input-Output Memory Management Unit [2], a RISC-V non-ISA specification.
MD	Memory domain.
MMIO	Memory mapped input/output devices.
NA4	Naturally aligned four-byte region, one of the address matching mode used in RISC-V PMP [1] and IOPMP.
NAPOT	Naturally aligned power-of-2 region, one of the address matching mode used in RISC-V PMP [1] and IOPMP.
N/A	Not available.
RRID	Request Role ID.
TOR	Top boundary of an arbitrary range, one of the address matching mode used in RISC-V PMP [1] and IOPMP.
WARL	Write any read legal, a behavior of fields.
RW1C	Read bit return the status and write '1' clear the status, a behavior of single bit fields.
W1CS	Write '1' clear and sticky to 0, a behavior of single bit fields: writing '1' clears the bit. If the bit is cleared, this state remains fixed until IOPMP registers are reset.
W1SS	Write '1' set and sticky to 1, a behavior of single bit fields: writing '1' sets the bit. If the bit is set, this state remains fixed until IOPMP registers are reset.
X(n)	The n-th register in the register array X, which starts from 0.
X[n]	The n-th bit of a register X or register field X
X($n : m$)	The n-th to m-th registers of a register X.
X[$n : m$]	The n-th to m-th bits of a register X or register field X.

2.1. Request-Role-ID and Transaction

Request Role ID (RRID) is a unique ID to identify a system-defined security context. For example, a unique RRID can be a transaction requester or a group of transaction requesters with the same permission. Depending on the IOPMP integration, transactions may be tagged with an RRID to identify the requester. Tagging requesters with RRID is implementation-dependent. The number of bits of an RRID is implementation-dependent as well. If different channels or modes of a requester could be granted different access permissions, they can have their own RRID.

2.2. Requester Port, Receiver Port and Control Port

An IOPMP has one or multiple requester ports, one or multiple receiver ports and one control port. A receiver port is where a transaction goes into the IOPMP, and a requester port is where a transaction leaves it if the transaction passes all the checks. The control port is used to program the IOPMP.

2.3. Memory Domain

A Request-Role-ID (RRID) serves as an abstract identifier for a transaction requester, representing one or more requesters that are assigned an identical set of potential access permissions. Conversely, a Memory Domain (MD) is an abstract construct representing a transaction destination. Its primary purpose is to logically group a set of memory regions that share a common functional or security objective, thereby simplifying the subsequent management. MDs are indexed starting from zero.

For example, a Network Interface Controller (NIC) typically utilizes multiple memory areas, such as a Receive (RX) data buffer, a Transmit (TX) data buffer, and a set of control registers. These distinct regions, due to their association with the single NIC device and similar access requirements, can be efficiently consolidated into a single Memory Domain. If a processor (or other bus agent) is given full control over the NIC's operation, that agent may then be explicitly associated with this MD.

It's important to note that, generally speaking, a single RRID can be associated with multiple memory domains (MDs), and a MD can be associated with multiple RRIDs. SRCMD Table is one of the fundamental structures in IOPMP to indicate associations between RRIDs and MDs, which will be introduced in SRCMD Table.

MDCFG Table also is one of fundamental structures in IOPMP to define a set of memory regions in a MD, which will be introduced in MDCFG Table. An IOPMP entry defines a single memory region, which will be introduced in IOPMP Entry and IOPMP Entry Array.

An RRID associated with a MD may not have full permissions on all memory regions of the MD, because the permission of each region is defined in the corresponding IOPMP entry.

2.4. IOPMP Entry and IOPMP Entry Array

The IOPMP entry array, contains `HWCFG1.entry_num` IOPMP entries, where the value zero is reserved. Each entry, starting from an index of zero, includes a specified memory region and the corresponding permissions.

For an entry indexed by *i*, `ENTRY_ADDR(i)`, `ENTRY_ADDRH(i)` and `ENTRY_CFG(i).a` encode the memory region in the same way as the RISC-V PMP. `ENTRY_ADDR(i)` and `ENTRY_ADDRH(i)` are the encoded address for the IOPMP with addresses greater than 34 bits. For addresses less than and equal to 34 bits, only `ENTRY_ADDR(i)` is in use. To determine whether `ENTRY_ADDRH(i)` are implemented, software can check `HWCFG0.addrh_en`. Please refer Hardware Configuration 0 for more details of `HWCFG0.addrh_en`. `ENTRY_CFG(i).a` is the address mode, which are OFF, NA4, NAPOT, and TOR. Please refer to the RISC-V privileged spec [1] for the details of the encoding schemes. `HWCFG0.tor_en` = 1 indicates the IOPMP supports TOR address mode.

Since the address encoding scheme of TOR refers to the previous entry's memory region, which is not in the same memory domain, it would cause unexpected results. If the first entry of a memory domain selects TOR, the entry refers to the previous memory domain. When the previous memory domain is changed unexpectedly, the region of this entry will be altered. To prevent the unexpected change of memory region, software should

- *avoid adopting TOR for the first entry of a memory domain; or*
- *set an OFF for the last entry of a memory domain with the maximal address during programming the IOPMP.*

`ENTRY_CFG(i).r/w/x` indicate the read access, write access, and instruction fetch permission and they are WARL. That is, an implementation can decide which bits are programmable or hardwired and which bit combinations are unwanted.

`ENTRY_CFG(i)` contains the basic permission fields for read, write, instruction fetch and address mode configuration.

Any entry with index ≥ `HWCFG1.entry_num` is not available. That is,

- Registers of the entry are not implemented.
- Address mode of the entry is treated as OFF when the IOPMP retrieves the entry in permission checks.

Memory domains are a way of dividing the IOPMP entry array into different subarrays. Each subarray is a memory domain. Each IOPMP entry can belong to at most one memory domain, while a memory domain could have multiple IOPMP entries.

A memory domain may have an IOPMP entry with index ≥ `HWCFG1.entry_num` due to its register encoding or implementation. The entry is not available.

When an RRID is associated with a memory domain, it is also inherently associated with all the entries that belong to that memory domain. Figure 2 illustrates an example of relations between RRIDs, memory domains, and IOPMP entries. In the example, RRID 0 is associated with entry 0, 1, 2, 5, 6, and RRID 1 is associated with entry 0, 1, 2, 3, 4.

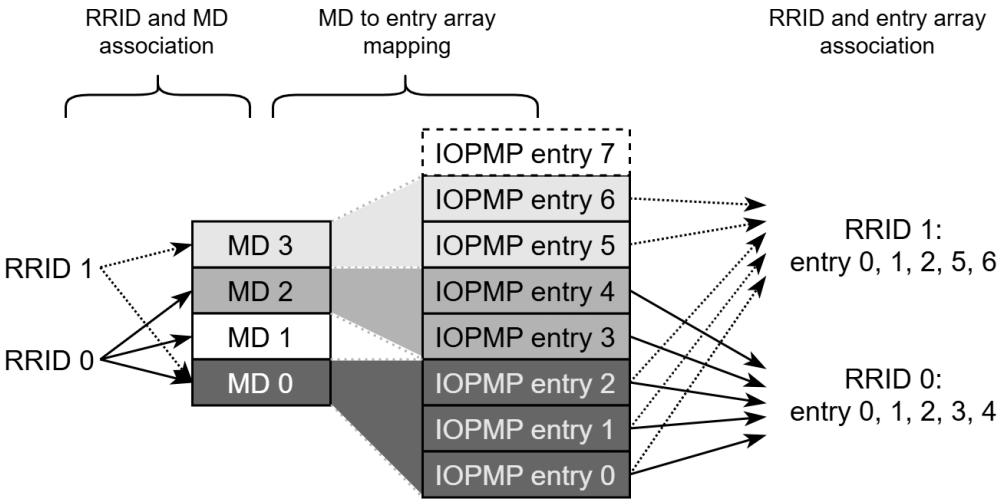

Figure 2. An example of relations between RRIDs, memory domains, and IOPMP entries.

2.5. SRCMD Table

The SRCMD Table is an array indexed by RRID that associates each requester with its memory domains. The table uses a bitmap approach where each RRID can be associated with multiple memory domains. When a IOPMP receives a transaction with RRID *s*, IOPMP first looks up the SRCMD Table to find out all the memory domains associated to requester *s*. An IOPMP instance can support up to 65,535 requesters, the actual number of requesters can be implementation-defined and is indicated in `HWCFG1.rrid_num`. The SRCMD Table defines the mapping from a transaction requester to its associated MDs.

The register `SRCMD_EN(s)` must be implemented for each existing RRID *s*. Every bit in the 31-bit field

`SRCMD_EN(s).md` indicates a memory domain. Bit *j* in `SRCMD_EN(s).md` indicates if MD *j* is associated with RRID *s*. If the number of MDs is more than 31, the register `SRCMD_ENH(s)` and its 32-bit field `SRCMD_ENH(s).mdh` can be implemented to support up to 63 memory domains. Bit *j* in `SRCMD_ENH(s).mdh` indicates if MD (*j* + 31) is associated with RRID *s*. For unimplemented memory domains, the corresponding bits should be hardwired read-only zero.

Every `SRCMD_EN(s)` has 1-bit field `l` for SRCMD Table protection. The bit is described in SRCMD Table Protection.

2.6. MDCFG Table

The MDCFG Table is used to map a memory domain to its own entries. It can be viewed as a partition of all entries in the IOPMP. The MDCFG defines which memory domain every entry belongs to. An entry can belong to at most one memory domain, but a memory domain can own zero or more entries. After retrieving all related entries, an IOPMP checks the transaction according to them.

In the MDCFG Table, every memory domain *m* has a register, `MDCFG(m)`. Field `MDCFG(m).t` represents the upper bound (not included) and field `MDCFG(m-1).t` represents the lower bound (included) for *m* > 0. They form the index range of IOPMP entries belonging to the memory domain *m*. That is,

- an entry with index *j* belongs to MD *m* if `MDCFG(m - 1).t` ≤ *j* < `MDCFG(m).t`, where *m* > 0.
- an entry with index *j* belongs to MD 0 if *j* < `MDCFG(0).t`.

Programmers should maintain proper settings in the MDCFG Table when `HWCFG0.enable` is set, where `MDCFG(m + 1).t` should be greater than or equal to `MDCFG(m).t` for any legal MD m. Otherwise, the MDCFG table has an improper setting, that is, there exist two legal MDs *m* and *k*, `MDCFG(k).t` > `MDCFG(m).t` for *k* < *m*. In this case, the specification denotes the MD *m* as having an improper setting.

When the MDCFG Table has an improper setting, the resulting association of IOPMP entries to MDs is implementation-dependent, but the following conditions must be held:

- an entry must belong to at most one memory domain; and
- the index of an entry in a lower-indexed memory domain must be lower than that in a higher-indexed memory domain.

For portability, programmers should ensure the MDCFG Table has a proper setting during the runtime. In the case that the process of programming or initializing the MDCFG Table causes an improper setting to occur transiently, software can deassociate effected MDs in SRCMD table before programming MDCFG table to avoid security issues. Even though the procedure may introduce extra steps, the table is not subject to change at such a high frequency as to hurt overall performance. Thus, the specification requires programmers to maintain proper settings instead of requiring specific hardware behaviors on improper settings.

2.7. Priority and Matching Logic

The IOPMP uses entry-based permission checking to determine whether transactions are allowed or denied.

When a transaction arrives at an IOPMP, the IOPMP first checks whether the RRID carried by the transaction is legal. If the RRID is illegal, the transaction is illegal with error type = "Unknown RRID" (0x06).

Whether an RRID is legal is implementation-dependent, even though it < `HWCFG1.rrid_num`.

IOPMP entries are prioritized according to their index values. Entries with lower indices are assigned a higher priority. When multiple entries match a transaction, the entry with the lowest index (highest priority) takes precedence.

The specification uses priority-based entry matching to provide deterministic behavior. Entries with lower indices have higher priority and take precedence when multiple entries match a transaction. Non-prioritized entries are supported for specific use cases, with their detailed behavior documented in the application notes.

An entry qualifies as a matching entry for an incoming transaction if:

- Its region covers any byte of the transaction,
- It is associated with the RRID carried by the transaction; and
- It holds the highest priority among entries that meet the previous criteria.

The matching entry can grant a transaction according to its access type. If the matching entry allows the access type, the transaction is legal. Every entry can permit read, write, and instruction fetch access by its `r`, `w`, and `x` bits, respectively.

The matching entry must match all bytes of a transaction, or the transaction is illegal with error type = "partial hit on a priority rule" (0x04), irrespective of its permission. If an entry is matched but doesn't grant the transaction permission to operate, the transaction is illegal with error type = "illegal read access" (0x01) for read access transaction, "illegal write access/AMO" (0x02) for write access/atomic memory operation (AMO) transaction, "illegal instruction fetch" (0x03) for instruction fetch transaction.

Finally, if no matching entry exists, the transaction is illegal with error type = "not hit any rule" (0x05).

To grant an AMO transaction permission, both read access permission and write access permission must be permitted.

Some AMO implementations of I/O agents are using a non-atomic read-modify-write sequence which could contain a read access transaction and a write access transaction, not a single AMO transaction. Therefore, IOPMP possibly captures error type = "illegal read access" (0x01) when read permission for the read-modify-write sequence from the I/O agents is not granted.

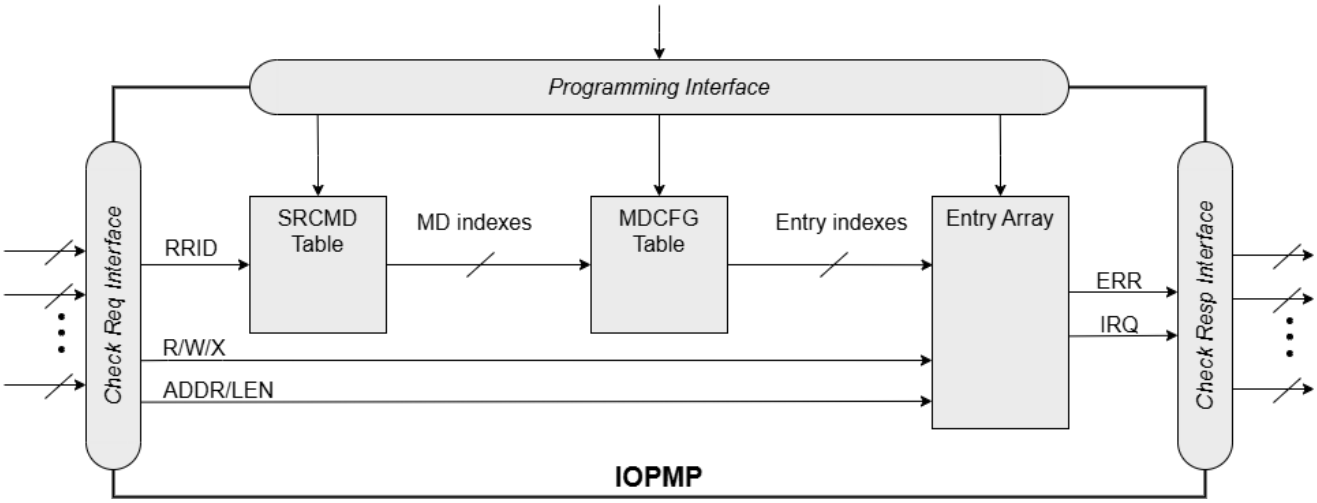

Figure 3. an example block diagram of an IOPMP. It illustrates the checking flow of an IOPMP. This IOPMP takes three inputs: RRID, the transaction type (read/write/instruction fetch), and the request range (address/len). It first looks up the SRCMD Table according to the RRID carried by the incoming transaction to retrieve associated MD indexes. By the MD indexes, the IOPMP looks up the MDCFG Table to get the belonging entry indexes. The final step checks the access right according to the above entry indexes and corresponding permissions. An interrupt, an error response, and/or a record is generated once the transaction fails the permission check in the step.

2.8. Error Reactions

Upon detecting an illegal transaction, the IOPMP could initiate one or more of the following actions:

- Trigger an interrupt to notify the system of the violation.
- Return implementation-defined bus error or faked succeed with an implementation-defined value from receiver ports.
- Log the error details in IOPMP error record registers.

IOPMP can trigger an interrupt on an access violation. The `ERR_CFG.ie` bit serves as the global interrupt enable configuration bit. The interrupt pending indication is equivalent to the error valid indication; both are flagged through the `ERR_INFO.v` bit. On an illegal transaction, an interrupt is triggered if the global interrupt is enabled (`ERR_CFG.ie` = 1).

Transactions that violate the IOPMP rule will by default yield a bus error. The bus error response behavior on an IOPMP violation can be configured globally via the `ERR_CFG` register. The IOPMP will signal the bus to the presence of a violation but will suppress the bus error if `ERR_CFG.rs` is implemented and set to 1 on a violation. User-defined suppression behavior allows, for example, a read response of 0x0.

For error type = "User-defined error" (0x0E, 0x0F), behavior of interrupt and error response is user-defined.

The error capture record maintains the specifics of the first illegal access detected, except for the condition:

- no interrupt regarding the access is triggered, and
- no bus error is returned.

An error capture only occurs when there is no pending error, that is, `ERR_INFO.v` = '0'. If a pending error exists (`v` = '1'), the record will not be updated, even if a new illegal access is detected. In other words, `v` indicates whether the content of the capture record is valid and should be intentionally cleared in order to capture subsequent illegal accesses. Software can write 1 to the bit to clear it. The error capture record is optional. If it is not implemented, `HWCFG0.no_err_rec` must be wired to one. It is possible to implement the

error capture record without the error entry index record (`ERR_REQID.eid`). In this case, `eid` must be wired to 0xffff.

The following table is a summary of error types:

Table 2. Summary of error types.

Error type	
0x00	No error
0x01	Illegal read access
0x02	Illegal write access/AMO
0x03	Illegal instruction fetch
0x04	Partial hit on a priority rule
0x05	Not hit any rule
0x06	Unknown RRID
0x07 ~ 0x0D	Reserved
0x0E ~ 0x0F	User-defined error

Chapter 3. Configuration Protection

Unauthorized alteration, even partial, of the IOPMP configuration poses a critical security threat, potentially leading to consequences ranging from Denial-of-Service (DoS) attacks to the wholesale compromise and leakage of sensitive data. Consequently, robust hardware mechanisms are indispensable for securing the integrity and content of the configuration registers.

This chapter details the configuration protection mechanisms provided by the IOPMP specification. These mechanisms allow implementations to prohibit modification of the configuration, either wholly or partially, based on the security requirements of the application scenario. Specifically, the IOPMP defines distinct write-protection controls for the SRCMD Table, the MDCFG Table, and the Entry Array.

The term 'lock' refers to a hardware feature that renders one or more fields or registers nonprogrammable until the IOPMP is reset. This feature serves to maintain the integrity of essential configurations in the event of a compromise of secure software. In cases where a lock bit is programmable, it is expected to be reset to '0' and is a W1SS field.

3.1. SRCMD Table Protection

Every `SRCMD_EN(s)` register has a bit `l` at bit 0, which is used to lock registers `SRCMD_EN(s)`, and `SRCMD_ENH(s)` if any.

The two fields `MDLCK.md` and `MDLCKH.mdh` have 63 bits together. Every bit is used to lock the MD association bits in the corresponding SRCMD Table entry. For MD $0 \le m \le 30$, `MDLCK.md[`m`]` locks `SRCMD_EN(s).md[`m`]` for all existing RRID s. For MD $31 \le m \le 62$, `MDLCKH.mdh[`m`]` locks `SRCMD_ENH(s).mdh[`m`]` for all existing RRID s.

Bit `MDLCK.l` is a sticky to 1 and indicates if `MDLCK` and `MDLCKH` are locked.

`MDLCK.md` is optional, if not implemented, `MDLCK.md` must be wired to 0 and `MDLCK.l` must be wired to 1.

`MDLCKH` is optional. It is available when `HWCFG0.md_num` > 31.

Locking SRCMD Table in either way can prevent the table from being altered accidentally or maliciously. By locking the association of the MD containing the configuration regions of a component, one can prevent the component from being configured by unwanted RRIDs. To make it more secure, one can use another high-priority MD containing the same regions but no permission, let it be associated with all unwanted RRIDs, and then lock the two MDs' associations by `MDLCK/MDLCKH`*. By adopting this approach, it is possible to safeguard the configuration from direct access by potentially compromised security software.*

3.2. MDCFG Table Protection

Register `MDCFGLCK` is designed to partially or fully lock the MDCFG Table. `MDCFGLCK` consists of two fields: `MDCFGLCK.l` and `MDCFGLCK.f`. `MDCFG(`m`)` is locked if $m <$ `MDCFGLCK.f`. `MDCFGLCK.f` is incremental-only. Any smaller value can not be written into it. The bit `MDCFGLCK.l` is used to lock `MDCFGLCK`.

If `MDCFG(`m`)` *is locked for MD m, while* `MDCFG(`m`-1)` *is not locked, it could lead to the potential addition or removal of unexpected IOPMP entries within the MD m. This can occur by manipulating* `MDCFG(`m`-1).t`*. Thus, the specification requires that if* `MDCFG(`m`)` *is locked for MD m, all its preceding MDCFG Table entries (*`MDCFG(0)` *to* `MDCFG(`m`-1)`*) must also be locked.*

3.3. Entry Protection

IOPMP entry protection is also related to the other IOPMP entries belonging to the same memory domain. For a MD, locked entries should be placed in the higher priorities. Otherwise, when the secure monitor is compromised, one unlocked entry in higher priority can override the effects of all the other locked or non-locked entries in lower priority. A register `ENTRYLCK` is defined to indicate the number of nonprogrammable entries. `ENTRYLCK` register has two fields: `ENTRYLCK.l` and `ENTRYLCK.f`. Any IOPMP entry with index i < `ENTRYLCK.f` is not programmable. `ENTRYLCK.f` is incremental-only. Any smaller value can not be written into it. Besides, `ENTRYLCK.l` is the lock to `ENTRYLCK.f` and itself. If `ENTRYLCK` is hardwired, `ENTRYLCK.l` must be wired to 1.

3.4. Summary of Table, Register, and Field Locks

Almost all programmable tables, registers, and fields have a corresponding lock mechanism that prevents updates until the IOPMP is reset. The following is a summary.

- SRCMD Table Protection - locks for the SRCMD Table.
- MDCFG Table Protection - locks for the MDCFG Table.
- Entry Protection - locks for the IOPMP entry array.

Error record registers do not have corresponding locks, as they are intended to be modified at runtime.

3.5. Prelocked Configurations

Prelocked configurations in the specification mean the fields or registers are locked right after reset. In practice, they could be hardwired and/or implemented by read-only memory. Every lock mechanism in this chapter can be optionally pre-locked. The non-zero reset value of `MDCFGLCK.f` reflects the pre-locked `MDCFG(j)`, where j< `MDCFGLCK.f`. The non-zero reset value of `ENTRYLCK.f` reflects the existing pre-locked entries. `SRCMD_EN(H)` can have prelocked bits fully or partially based on presets of `MDLCK.md` and `SRCMD_EN.l`. Please refer Summary of Table, Register, and Field Locks for all lock bits.

Chapter 4. Registers

If an optional register is not implemented, the behavior is implementation-dependent unless otherwise specified. An optional field in an implemented register means being WARL. If it is not programmable, it should be hardwired to value matching its meaning and should not cause any effect when written. Reserved regions are reserved for standard use. The behavior in the reserved regions is implementation-dependent.

Table 3. Register summary

OFFSET	Register	Description
INFO		
0x0000	`VERSION`	Indicates the specification and the IP vendor.
0x0004	`IMPLEMENTATION`	Indicates the implementation version.
0x0008	`HWCFG0`	Indicates the basic configurations of current IOPMP instance.
0x000C	`HWCFG1`	Indicates the supported numbers of RRIDs and entries.
0x0010	`HWCFG2`	(Optional) indicates the extended configurations of current IOPMP instance for extension.
0x0014	`HWCFG3`	(Optional) indicates the extended configurations of current IOPMP instance for application note.
0x0028	`HWCFG_USER`	(Optional) user-defined configurations of current IOPMP instance.
0x002C	`ENTRYOFFSET`	Indicates the internal address offsets of each table.
Configuration Protection		
0x0040	`MDLCK`	Lock register for lower MDs in SRCMD Table.
0x0044	`MDLCKH`	(Optional) lock register for higher MDs in SRCMD Table.
0x0048	`MDCFGLCK`	Lock register for MDCFG Table.
0x004C	`ENTRYLCK`	Lock register for IOPMP entry array.
Error Reporting		
0x0060	`ERR_CFG`	Indicates the reactions for the violations.
0x0064	`ERR_INFO`	Indicates the information regarding captured violations.
0x0068	`ERR_REQADDR`	Indicates lower request address regarding the first captured violation.
0x006C	`ERR_REQADDRH`	(Optional) indicates higher request address regarding the first captured violation.
0x0070	`ERR_REQID`	Indicates the RRID and entry index regarding the first captured violation.
0x0080 ~ 0x009F	`ERR_USER(0)` ~ `ERR_USER(7)`	(Optional) user-defined violation information.
MDCFG Table, m = 0…`HWCFG0.md_num`-1		
0x0800 + (m) × 4	`MDCFG(`m`)`	MD config register, which specifies the indices of IOPMP entries belonging to a MD.
SRCMD Table, s = 0…`HWCFG1.rrid_num`-1		
0x1000 + (s) × 32	`SRCMD_EN(s)`	The bitmapped MD enabling register of the RRID s that `SRCMD_EN(s)[`m`]` indicates if the RRID is associated with MD m.
0x1004 + (s) × 32	`SRCMD_ENH(s)`	(Optional) The bitmapped MD enabling register of the RRID s that `SRCMD_ENH(s)[`m`]` indicates if the RRID is associated with MD (m+31).

OFFSET	Register	Description
Entry Array, *i* =0…`HWCFG1.entry_num`-1		
`ENTRYOFFSET` + (*i*) × 16	`ENTRY_ADDR(i)`	Bit 33:2 of address (region) for entry *i*.
`ENTRYOFFSET` + 0x4 + (*i*) × 16	`ENTRY_ADDRH(i)`	(Optional) bit 65:34 of the address (region) for entry *i*.
`ENTRYOFFSET` + 0x8 + (*i*) × 16	`ENTRY_CFG(i)`	The configuration of entry *i*.
`ENTRYOFFSET` + 0xC + (*i*) × 16	`ENTRY_USER_CFG(i)`	(Optional) user-defined attributes.

Other regions in offset 0x0000 to (0x1000 + `HWCFG1.rrid_num` × 32) are reserved.

4.1. INFO registers

INFO registers are used to indicate the IOPMP instance configuration info.

4.1.1. Version Number and Vendor ID (`VERSION`)

Field	Bits	R/W	Default	Description
`vendor`	23:0	R	IMP	The JEDEC manufacturer ID.
`specver`	31:24	R	IMP	The specification version. Bits 27:24 hold the major version and bits 31:28 hold the minor version. For example, version 1.0 is reported as 0x10.

4.1.2. Implementation ID (`IMPLEMENTATION`)

Field	Bits	R/W	Default	Description
`impid`	31:0	R	IMP	The user-defined implementation ID.

4.1.3. Hardware Configuration 0 (`HWCFG0`)

Field	Bits	R/W	Default	Description
`enable`	0:0	W1SS	IMP	Indicate if the IOPMP checks transactions by default. If it is programmable, it must be initialized to 0 and sticky to 1. If it is not programmable, it must be wired to 1.
`HWCFG2_en`	1:1	R	IMP	Indicate if `HWCFG2` is implemented.
`HWCFG3_en`	2:2	R	IMP	Indicate if `HWCFG3` is implemented.
`rsv`	22:3	ZERO	0	Must be zero on write, reserved for future.
`no_err_rec`	23:23	R	IMP	Indicate if error record (`ERR_INFO`, `ERR_REQADDR`, `ERR_REQADDRH`, `ERR_REQID` and `ERR_USER(0)` ~ `ERR_USER(7)`) is not implemented.
`md_num`	29:24	R	IMP	Indicate the supported number of MD in the instance.

Field	Bits	R/W	Default	Description
addrh_en	30:30	R	IMP	Indicate if ENTRY_ADDRH(*i*) and ERR_REQADDRH are available.
tor_en	31:31	R	IMP	Indicate if TOR is supported.

4.1.4. Hardware Configuration 1 (HWCFG1)

Field	Bits	R/W	Default	Description
rrid_num	15:0	R	IMP	Indicate the supported number of RRID in the instance
entry_num	31:16	R	IMP	Indicate the supported number of entries in the instance, which should be larger than zero.

4.1.5. Hardware Configuration 2 (HWCFG2)

See Chapter 5, *Extensions*.

4.1.6. Hardware Configuration 3 (HWCFG3)

See Appendix A, *Application Note*.

4.1.7. User-defined Hardware Configuration (HWCFG_USER)

HWCFG_USER is an optional register to provide users to define their own configurations.

Field	Bits	R/W	Default	Description
user	31:0	IMP	IMP	(Optional) user-defined registers

4.1.8. Entry Array Offset (ENTRYOFFSET)

Field	Bits	R/W	Default	Description
offset	31:0	R	IMP	Indicate the offset address of the IOPMP array from the base of an IOPMP instance, a.k.a. the address of VERSION. Note: the offset is a signed number in two's complement representation. That is, the IOPMP array can be placed in front of VERSION.

4.2. Configuration Protection Registers

4.2.1. Memory Domain Lock (MDLCK and MDLCKH)

MDLCK is a register with a bitmap field to indicate which lower MDs are locked in the SRCMD Table.

Field	Bits	R/W	Default	Description
l	0:0	W1SS	IMP	Lock bit to MDLCK and MDLCKH register.
md	31:1	WARL	IMP	md[*m*] is sticky to 1 and indicates if SRCMD_EN(*s*).md[*m*] is locked for all RRID *s*.

MDLCKH is a register with a bitmap field to indicate which higher MDs are locked in the SRCMD Table.

Field	Bits	R/W	Default	Description
mdh	31:0	WARL	IMP	mdh[*m*] is sticky to 1 and indicates if SRCMD_ENH(*s*).mdh[*m*] is locked for all RRID *s*.

4.2.2. MDCFG Table Lock (MDCFGLCK)

MDCFGLCK is the lock register to MDCFG Table.

Field	Bits	R/W	Default	Description
l	0:0	W1SS	IMP	Lock bit to MDCFGLCK register.
f	6:1	WARL	IMP	Indicate the number of locked MDCFG entries - MDCFG(*m*) is locked for *m* < f. On write, the field only accepts the value larger than the previous value until the next reset cycle; otherwise, there is no effect. If f > HWCFG0.md_num, all MDCFG entries are locked.
rsv	31:7	ZERO	0	Must be zero on write, reserved for future

4.2.3. Entry Array Lock (ENTRYLCK)

ENTRYLCK is the lock register to entry array.

Field	Bits	R/W	Default	Description
l	0:0	W1SS	IMP	Lock bit to ENTRYLCK register.
f	16:1	WARL	IMP	Indicate the number of locked IOPMP entries - ENTRY_ADDR(*i*), ENTRY_ADDRH(*i*), and ENTRY_CFG(*i*) are locked for *i* < f. On write, the field only accepts the value larger than the previous value until the next reset cycle; otherwise, there is no effect. If f > HWCFG1.entry_num, all IOPMP entries are locked.
rsv	31:17	ZERO	0	Must be zero on write, reserved for future

4.3. Error Capture Registers

4.3.1. Error Configuration (ERR_CFG)

ERR_CFG is a read/write WARL register used to configure the global error reporting behavior on an IOPMP violation.

Field	Bits	R/W	Default	Description
l	0:0	W1SS	IMP	Lock bit to ERR_CFG register.
ie	1:1	WARL	IMP	Enable the interrupt of the IOPMP rule violation.
rs	2:2	WARL	IMP	To suppress an error response on an IOPMP rule violation. • 0x0: respond an implementation-dependent error, such as a bus error • 0x1: respond a success with a pre-defined value to the requester instead of an error
rsv	31:3	ZERO	0	Must be zero on write, reserved for future

4.3.2. Error Information (`ERR_INFO`)

`ERR_INFO` captures more detailed error information.

Field	Bits	R/W	Default	Description
`v`	0:0	RW1C	0	Indicate if the illegal capture recorder (`ERR_REQID`, `ERR_REQADDR`, `ERR_REQADDRH`, `ERR_INFO.ttype`, and `ERR_INFO.etype`) has a valid content and will keep the content until the bit is cleared. An interrupt will be triggered if a violation is detected and related interrupt enable/suppression configure bits are not disabled, the interrupt will keep asserted until the error valid is cleared. Write 1 clears the bit, the illegal recorder reactivates and the interrupt (if enabled). Write 0 causes no effect on the bit.
`ttype`	2:1	R	0	Indicated the transaction type of the first captured violation • 0x00 = reserved • 0x01 = read access • 0x02 = write access/AMO • 0x03 = instruction fetch
`etype`	7:4	R	0	Indicated the type of violation • 0x00 = no error • 0x01 = illegal read access • 0x02 = illegal write access/AMO • 0x03 = illegal instruction fetch • 0x04 = partial hit on a priority rule • 0x05 = not hit any rule • 0x06 = unknown RRID • 0x07 ~ 0x0D = reserved • 0x0E ~ 0x0F = user-defined error
`rsv`	31:8	ZERO	0	Must be zero on write, reserved for future

When the bus matrix doesn't have a signal to indicate an instruction fetch, the `ttype` and `etype` can never return "instruction fetch" (0x03) and "illegal instruction fetch" (0x03), respectively.

4.3.3. Errored Request Address (`ERR_REQADDR` and `ERR_REQADDRH`)

`ERR_REQADDR` indicates the lower errored request address of the first captured violation.

Field	Bits	R/W	Default	Description
`addr`	31:0	R	DC	Indicate the errored address[33:2]

`ERR_REQADDRH` indicates the higher errored request address of the first captured violation.

Field	Bits	R/W	Default	Description
`addrh`	31:0	R	DC	Indicate the errored address[65:34]

4.3.4. Errored Request IDs (`ERR_REQID`)

`ERR_REQID` indicates the errored RRID and entry index of the first captured violation.

Field	Bits	R/W	Default	Description
`rrid`	15:0	R	DC	Indicate the errored RRID.
`eid`	31:16	R	DC	Indicates the index pointing to the entry that catches the violation. If no entry is hit, for example, `ERR_INFO.etype`= 0x05 or 0x06, the value of this field is invalid. If the field is not implemented, it must be wired to 0xffff.

4.3.5. User-defined Error Information (`ERR_USER(0)` ~ `ERR_USER(7)`)

`ERR_USER(0)` ~ `ERR_USER(7)` are optional registers to provide users to define their own error capture information.

Field	Bits	R/W	Default	Description
`user`	31:0	IMP	IMP	(Optional) user-defined registers

4.4. MDCFG Table Registers

MDCFG Table is a lookup to specify the number of IOPMP entries that is associated with each MD.

4.4.1. MD *m* Configuration (`MDCFG(m)`)

Field	Bits	R/W	Default	Description
`t`	15:0	WARL	DC/IMP	Indicate the encoding the range of memory domain *m*. An IOPMP entry with index *j* belongs to MD *m* • If `MDCFG(m-1).t` ≤ *j* < `MDCFG(m).t`, where m > 0. • If *j* < `MDCFG(0).t` where m = 0. • If `MDCFG(m-1).t` > `MDCFG(m).t`, then MDCFG Table is improperly programmed. Please refer MDCFG Table for IOPMP behavior of improperly programming.
`rsv`	31:16	ZERO	0	Must be zero on write, reserved for future.

4.5. SRCMD Table Registers

4.5.1. RRID *s* MD Enabling (`SRCMD_EN(s)` and `SRCMD_ENH(s)`)

`SRCMD_EN(s)` is the SRCMD Table register of RRID *s* for lower MDs, *s* = 0...`HWCFG1.rrid_num`-1.

Field	Bits	R/W	Default	Description
`l`	0:0	W1SS	IMP	A sticky lock bit. When set, locks `SRCMD_EN(s)` and `SRCMD_ENH(s)`.
`md`	31:1	WARL	DC	`md[m]` = 1 indicates MD *m* is associated with RRID *s*.

`SRCMD_ENH(s)` is the SRCMD Table register of RRID *s* for higher MDs, *s* = 0...`HWCFG1.rrid_num`-1.

Field	Bits	R/W	Default	Description
mdh	31:0	WARL	DC	mdh[*m*] = 1 indicates MD (*m*+31) is associated with RRID *s*.

4.6. Entry Array Registers

4.6.1. Entry *i* Address (ENTRY_ADDR(*i*) and ENTRY_ADDRH(*i*))

A complete 64-bit address consists of these two registers, ENTRY_ADDR and ENTRY_ADDRH. However, an IOPMP can only manage a segment of space, so an implementation would have a certain number of the most significant bits that are the same among all entries. These bits are allowed to be hardwired.

ENTRY_ADDR(*i*) is bit 33:2 of address (region) for entry *i*, *i* =0…HWCFG1.entry_num-1

Field	Bits	R/W	Default	Description
addr	31:0	WARL	DC	The physical address[33:2] of protected memory region.

ENTRY_ADDRH(*i*) is bit 65:34 of address (region) for entry *i*, *i* =0…HWCFG1.entry_num-1

Field	Bits	R/W	Default	Description
addrh	31:0	WARL	DC	The physical address[65:34] of protected memory region.

4.6.2. Entry *i* Configuration (ENTRY_CFG(*i*))

Field	Bits	R/W	Default	Description
r	0:0	WARL	DC	The read permission to protected memory region
w	1:1	WARL	DC	The write permission to the protected memory region
x	2:2	WARL	DC	The instruction fetch permission to the protected memory region
a	4:3	WARL	DC	The address mode of the IOPMP entry • 0x0: OFF • 0x1: TOR • 0x2: NA4 • 0x3: NAPOT
rsv	31:5	ZERO	0	Must be zero on write, reserved for future

Bits r, w, and x, grant read, write, or instruction fetch permission, respectively. Not each bit should be programmable. Some or all of them could be wired. Besides, an implementation can optionally impose constraints on their combinations.

4.6.3. User-defined Entry *i* Configuration (ENTRY_USER_CFG(*i*))

ENTRY_USER_CFG are implementation defined registers that allows users to define their own additional IOPMP check rules beside the rules.

Field	Bits	R/W	Default	Description
`im`	31:0	IMP	IMP	User customized field

Chapter 5. Extensions

This chapter presents IOPMP extensions designed for specific use cases. The features described in this chapter are optional extensions to the baseline IOPMP specification. Implementations are not required to support these extensions. However, an implementation that claims conformance to an extension MUST adhere to the definitions specified herein. Section 5.1 describes all relevant registers. The subsequent sections describe each IOPMP extension in detail.

5.1. Registers

Table 4, Table 5, and Table 6 provide summaries of registers, fields, and their corresponding sections.

This chapter also introduces an additional error type (`ERR_INFO.etype`). Table 7 lists all error types defined in both the baseline specification and the extensions.

Table 4. Register and field summary of extensions between offset 0x0000 and 0x07FF.

Offset	Register	Field	Bits	Related section(s)
0x0010	`HWCFG2`	`prio_entry`	15:0	Section 5.3, "Non-priority IOPMP entries"
		`prio_ent_prog`	16:16	
		`non_prio_en`	17:17	
		`rsv`	25:18	(Must be zero on write, reserved for future.)
		`msi_en`	26:26	Section 5.6, "Message-Signaled Interrupts (MSI) Extension"
		`peis`	27:27	Section 5.4, "Per-entry Interrupt/Bus Error Suppression"
		`pees`	28:28	
		`sps_en`	29:29	Section 5.2, "Secondary Permission Setting (SPS)"
		`stall_en`	30:30	Section 5.7, "Safe Runtime Configuration"
		`mfr_en`	31:31	Section 5.5, "Multi-Faults Record"
0x0030	`MDSTALL`	All		Section 5.7, "Safe Runtime Configuration"
0x0034	`MDSTALLH`	All		
0x0038	`RRIDSCP`	All		
0x0060	`ERR_CFG`	`msi_sel`	3:3	Section 5.6, "Message-Signaled Interrupts (MSI) Extension"
		`stall_violation_en`	4:4	Section 5.7, "Safe Runtime Configuration"
		`msidata`	18:8	Section 5.6, "Message-Signaled Interrupts (MSI) Extension"
0x0064	`ERR_INFO`	`msi_werr`	3:3	Section 5.6, "Message-Signaled Interrupts (MSI) Extension"
		`etype`	7:4	Section 5.7, "Safe Runtime Configuration"
		`svc`	8:8	Section 5.5, "Multi-Faults Record"
0x0074	`ERR_MFR`	All		Section 5.5, "Multi-Faults Record"
0x0078	`ERR_MSIADDR`	All		Section 5.6, "Message-Signaled Interrupts (MSI) Extension"
0x007C	`ERR_MSIADDRH`	All		

Table 5. Register summary of extensions in SRCMD Table.

SRCMD Table, s = 0…`HWCFG1.rrid_num`-1.		
Offset	Register	Related section(s)
0x1000 + (s) × 32	`SRCMD_R(s)`	Section 5.2, "Secondary Permission Setting (SPS)"
0x100C + (s) × 32	`SRCMD_RH(s)`	
0x1010 + (s) × 32	`SRCMD_W(s)`	
0x1014 + (s) × 32	`SRCMD_WH(s)`	
0x1018 + (s) × 32	`SRCMD_X(s)`	
0x101C + (s) × 32	`SRCMD_XH(s)`	

Table 6. Register and field summary of extensions in entry array.

Entry Array, i = 0…`HWCFG1.entry_num`-1				
Offset	Register	Field	Bits	Related section(s)
`ENTRYOFFSET` + 0x8 + (i) × 16	`ENTRY_CFG(i)`	`sire`	5:5	Section 5.4, "Per-entry Interrupt/Bus Error Suppression"
		`siwe`	6:6	
		`sixe`	7:7	
		`sere`	8:8	
		`sewe`	9:9	
		`sexe`	10:10	

Table 7. All error types of the baseline specification and extension.

Error type		Related section(s)
0x00	No error	(baseline specification)
0x01	Illegal read access	
0x02	Illegal write access/AMO	
0x03	Illegal instruction fetch	
0x04	Partial hit on a priority rule	
0x05	Not hit any rule	
0x06	Unknown RRID	
0x07	Error due to a stalled transaction	Section 5.7, "Safe Runtime Configuration"
0x08 ~ 0x0D	Reserved	(baseline specification)
0x0E ~ 0x0F	User-defined error	

5.1.1. Hardware Configuration 2 (`HWCFG2`)

Field	Bits	R/W	Default	Description
`prio_entry`	15:0	WARL	IMP	Indicate the number of entries matched with priority. These rules should be placed in the lowest order. Within these rules, the lower order has a higher priority. If `HWCFG2.non_prio_en` is 0, the default value of `prio_entry` is DC.

Field	Bits	R/W	Default	Description
`prio_ent_prog`	16:16	W1CS	IMP	A write-1-clear bit is sticky to 0 and indicates if `HWCFG2.prio_entry` is programmable. Reset to 1 if the implementation supports programmable `prio_entry`, otherwise, wired to 0. If `HWCFG2.non_prio_en` is 0, the default value of `prio_ent_prog` is DC.
`non_prio_en`	17:17	R	IMP	Indicates whether the IOPMP supports non-priority entries.
`rsv`	25:18	ZERO	IMP	Must be zero on write, reserved for future.
`msi_en`	26:26	R	IMP	Indicates whether the IOPMP supports MSI extension.
`peis`	27:27	R	IMP	Indicate if the IOPMP implements interrupt suppression per entry, including fields `sire` and `siwe` in `ENTRY_CFG(i)`, i = 0... `HWCFG1.entry_num`-1.
`pees`	28:28	R	IMP	Indicate if the IOPMP implements the error suppression per entry, including fields `sere` and `sewe` in `ENTRY_CFG(i)`, i = 0... `HWCFG1.entry_num`-1.
`sps_en`	29:29	R	IMP	Indicates secondary permission settings are supported; which are `SRCMD_R/RH(s)`, `SRCMD_W/WH(s)`, and `SRCMD_X/XH(s)` registers, s = 0 ... `HWCFG1.rrid_num` - 1.
`stall_en`	30:30	R	IMP	Indicate if the IOPMP implements stall-related features, which are `MDSTALL`, `MDSTALLH`, `RRIDSCP`, and `ERR_CFG.stall_violation_en`.
`mfr_en`	31:31	R	IMP	Indicate if the IOPMP implements Multi Faults Record, that is `ERR_MFR` and `ERR_INFO.svc`.

5.1.2. MD-Based Stall Control (`MDSTALL` and `MDSTALLH`)

`MDSTALL` is optional and used to support atomicity issue while programming the IOPMP, as the IOPMP rule may not be updated in a single transaction.

Field	Bits	R/W	Default	Description
`exempt`	0:0	W	N/A	Stall transactions from selected RRIDs: • 0: the RRIDs are associated with a selected MD • 1: the RRIDs are not associated with any selected MD Please refer Stall transactions for detailed operations
`is_busy`	0:0	R	0	Indicates the status of previous writing `MDSTALL` and `RRIDSCP`: • 0: the write has taken effect or no previous write • 1: the write has not taken effect
`md`	31:1	WARL	0	Writing `md[m]`=1 selects MD m; reading `md[m]` = 1 means MD m selected.

`MDSTALLH` is optional and is a higher MD part of `MDSTALL`.

Field	Bits	R/W	Default	Description
`mdh`	31:0	WARL	0	Writing `mdh[m]`=1 selects MD (m+31); reading `mdh[m]` = 1 means MD (m+31) selected.

5.1.3. RRID Cherry Pick Stall Control (RRIDSCP)

RRIDSCP is optional and used to support fine-grained control for MD-Based Stall Control.

Field	Bits	R/W	Default	Description
rrid	15:0	WARL	DC	RRID to select.
rsv	29:16	ZERO	0	Must be zero on write, reserved for future.
op	31:30	W	N/A	• 0: query • 1: stall transactions associated with selected RRID • 2: don't stall transactions associated with selected RRID • 3: reserved
stat	31:30	R	0	• 0: RRIDSCP is not implemented • 1: transactions associated with selected RRID are stalled • 2: transactions associated with selected RRID are not stalled • 3: unimplemented or unselectable RRID

5.1.4. Error Configuration (ERR_CFG) with All Extensions

ERR_CFG is a read/write WARL register used to configure the global error reporting behavior on an IOPMP violation.

Field	Bits	R/W	Default	Description
l	0:0	W1SS	IMP	Lock bit to ERR_CFG register.
ie	1:1	WARL	IMP	Enable the interrupt of the IOPMP rule violation.
rs	2:2	WARL	IMP	To suppress an error response on an IOPMP rule violation. • 0x0: respond an implementation-dependent error, such as a bus error • 0x1: respond a success with a pre-defined value to the requester instead of an error
msi_sel	3:3	WARL	IMP	Indicates whether the IOPMP triggers interrupt by MSI or wired interrupt: • 0x0: the IOPMP triggers interrupt by wired interrupt • 0x1: the IOPMP triggers interrupt by MSI
stall_violation_en	4:4	WARL	IMP	Indicates whether the IOPMP faults stalled transactions. When the bit is set, the IOPMP faults the transactions if the corresponding RRID is not exempt from stall.
rsv1	7:5	ZERO	0	Must be zero on write, reserved for future
msidata	18:8	WARL	IMP	The data to trigger MSI
rsv2	31:19	ZERO	0	Must be zero on write, reserved for future

5.1.5. Error Information (ERR_INFO) with All Extensions

ERR_INFO captures more detailed error information.

Field	Bits	R/W	Default	Description
`v`	0:0	RW1C	0	Indicate if the illegal capture recorder (`ERR_REQID`, `ERR_REQADDR`, `ERR_REQADDRH`, `ERR_INFO.ttype`, and `ERR_INFO.etype`) has a valid content and will keep the content until the bit is cleared. An interrupt will be triggered if a violation is detected and related interrupt enable/suppression configure bits are not disabled, the interrupt will keep asserted until the error valid is cleared. Write 1 clears the bit, the illegal recorder reactivates and the interrupt (if enabled). Write 0 causes no effect on the bit.
`ttype`	2:1	R	0	Indicated the transaction type of the first captured violation • 0x00 = reserved • 0x01 = read access • 0x02 = write access/AMO • 0x03 = instruction fetch
`msi_werr`	3:3	RW1C	0	It's asserted when the write access to trigger an IOPMP originated MSI has failed. When it's not available, it must be ZERO. Write 1 clears the bit. Write 0 causes no effect on the bit.
`etype`	7:4	R	0	Indicated the type of violation • 0x00 = no error • 0x01 = illegal read access • 0x02 = illegal write access/AMO • 0x03 = illegal instruction fetch • 0x04 = partial hit on a priority rule • 0x05 = not hit any rule • 0x06 = unknown RRID • 0x07 = error due to a stalled transaction. It should not happen when `ERR_CFG.stall_violation_en` is 0. • 0x08 ~ 0x0D = N/A, reserved for future • 0x0E ~ 0x0F = user-defined error
`svc`	8:8	R	0	Indicate there is a subsequent violation caught in `ERR_MFR`. Implemented only for `HWCFG2.mfr_en`=1, otherwise, ZERO.
`rsv`	31:9	ZERO	0	Must be zero on write, reserved for future

5.1.6. Multi-Faults Record (`ERR_MFR`)

Field	Bits	R/W	Default	Description
`svw`	15:0	R	DC	Subsequent violations in the window indexed by `svi`. `svw[`*j*`]`=1 for the at least one subsequent violation issued from RRID= `svi` * 16 + *j*.

Field	Bits	R/W	Default	Description
`svi`	27:16	WARL	0	Window's index to search subsequent violations. When read, IOPMP sequentially scans all windows from `svi` until one subsequent violation is found. Once the last available window is scanned, the next window to be scanned is the first record window (index is 0). `svi` indexes the found subsequent violation or `svi` has been rounded back to the same value. After read, the window's content, `svw`, should be clean.
`rsv`	30:28	ZERO	0	Must be zero on write, reserved for future.
`svs`	31:31	R	0	The status of this window's content: • 0x0 : no subsequent violation found • 0x1 : subsequent violation found

5.1.7. MSI Message Address (`ERR_MSIADDR` and `ERR_MSIADDRH`)

`ERR_MSIADDR` indicates the lower MSI message address for error reactions.

Field	Bits	R/W	Default	Description
`msiaddr`	31:0	WARL	IMP	The address to trigger MSI. For `HWCFG0.addrh_en`=0, it contains bits 33 to 2 of the address; otherwise, it contains bits 31 to 0. Available only if `HWCFG2.msi_en`=1.

`ERR_MSIADDRH` indicates the higher MSI message address for error reactions.

Field	Bits	R/W	Default	Description
`msiaddrh`	31:0	WARL	IMP	The higher 32 bits of the address to trigger MSI. Available only if `HWCFG0.addrh_en`=1 and `HWCFG2.msi_en`=1.

5.1.8. RRID *s* Read Permission (`SRCMD_R(s)` and `SRCMD_RH(s)`)

`SRCMD_R(s)` is the read permission register of RRID *s* for lower MDs, *s* = 0...`HWCFG1.rrid_num`-1.

Field	Bits	R/W	Default	Description
`rsv`	0:0	ZERO	0	Must be zero on write, reserved for future.
`md`	31:1	WARL	DC	`md[`*m*`]` = 1 indicates RRID *s* has read access and instruction fetch permission to the corresponding MD *m*.

`SRCMD_RH(s)` is the read permission register of RRID *s* for higher MDs, *s* = 0...`HWCFG1.rrid_num`-1.

Field	Bits	R/W	Default	Description
`mdh`	31:0	WARL	DC	`mdh[`*m*`]` = 1 indicates RRID *s* has read access and instruction fetch permission to MD (*m*+31).

5.1.9. RRID *s* Write Permission (`SRCMD_W(s)` and `SRCMD_WH(s)`)

`SRCMD_W(s)` is the write permission register of RRID *s* for lower MDs, *s* = 0...`HWCFG1.rrid_num`-1.

Field	Bits	R/W	Default	Description
`rsv`	0:0	ZERO	0	Must be zero on write, reserved for future.
`md`	31:1	WARL	DC	`md[`*m*`]` = 1 indicates RRID *s* has write permission to the corresponding MD *m*.

`SRCMD_WH(s)` is the write permission register of RRID *s* for higher MDs, *s* = 0...`HWCFG1.rrid_num`-1.

Field	Bits	R/W	Default	Description
`mdh`	31:0	WARL	DC	`mdh[`*m*`]` = 1 indicates RRID *s* has write permission to MD (*m*+31).

5.1.10. RRID *s* Instruction Fetch Permission (`SRCMD_X(s)` and `SRCMD_XH(s)`)

`SRCMD_X(s)` is the instruction fetch permission register of RRID *s* for lower MDs, *s* = 0...`HWCFG1.rrid_num`-1.

Field	Bits	R/W	Default	Description
`rsv`	0:0	ZERO	0	Must be zero on write, reserved for future.
`md`	31:1	WARL	DC	`md[`*m*`]` = 1 indicates RRID *s* has instruction fetch permission to the corresponding MD *m*.

`SRCMD_XH(s)` is the instruction fetch permission register of RRID *s* for higher MDs, *s* = 0...`HWCFG1.rrid_num` -1.

Field	Bits	R/W	Default	Description
`mdh`	31:0	WARL	DC	`mdh[`*m*`]` = 1 indicates RRID *s* has instruction fetch permission to MD (*m*+31).

5.1.11. Entry *i* Configuration (`ENTRY_CFG(i)`) with All Extensions

Field	Bits	R/W	Default	Description
`r`	0:0	WARL	DC	The read permission to protected memory region
`w`	1:1	WARL	DC	The write permission to the protected memory region
`x`	2:2	WARL	DC	The instruction fetch permission to the protected memory region
`a`	4:3	WARL	DC	The address mode of the IOPMP entry • 0x0: OFF • 0x1: TOR • 0x2: NA4 • 0x3: NAPOT
`sire`	5:5	WARL	IMP	Suppress interrupt for an illegal read access caught by the entry.
`siwe`	6:6	WARL	IMP	Suppress interrupt for an illegal write access/AMO caught by the entry.
`sixe`	7:7	WARL	IMP	Suppress interrupt on an illegal instruction fetch caught by the entry.

Field	Bits	R/W	Default	Description
`sere`	8:8	WARL	IMP	Suppress the (bus) error on an illegal read access caught by the entry: • 0x0: respond an error if `ERR_CFG.rs` is 0x0. • 0x1: do not respond an error. User to define the behavior, e.g., respond a success with an implementation-dependent value to the requester.
`sewe`	9:9	WARL	IMP	Suppress the (bus) error on an illegal write access/AMO caught by the entry: • 0x0: respond an error if `ERR_CFG.rs` is 0x0. • 0x1: do not respond an error. User to define the behavior, e.g., respond a success if response is needed
`sexe`	10:10	WARL	IMP	Suppress the (bus) error on an illegal instruction fetch caught by the entry: • 0x0: respond an error if `ERR_CFG.rs` is 0x0. • 0x1: do not respond an error. User to define the behavior, e.g., respond a success with an implementation-dependent value to the requester.
`rsv`	31:11	ZERO	0	Must be zero on write, reserved for future

5.2. Secondary Permission Setting (SPS)

This extension provides a mechanism to reduce the consumption of IOPMP entries in systems where multiple requesters (RRIDs) require different access permissions to the same shared memory regions.

Motivation: In the baseline specification, if RRID 'A' requires Read-Write (RW) access to a region and RRID 'B' requires Read-Only (RO) access to the exact same address range, two separate IOPMP entries must be configured (typically in two different Memory Domains). This consumes valuable Entry Array resources simply to define duplicate address ranges with different permissions.

The SPS extension addresses this by adding a second layer of permission checking within the SRCMD Table. This allows a single, shared IOPMP entry to define the address region with a superset of permissions (e.g., both read and write permissions are allowed), while the additional permission registers restrict the permissions for each RRID individually. This allows multiple RRIDs to share a single Memory Domain and its corresponding entries, significantly conserving the total number of entries required.

The field `sps_en` in register `HWCFG2` Hardware Configuration 2 indicates if the IOPMP implements the SPS extension.

When the SPS extension is implemented, each SRCMD Table entry additionally defines permission registers: `SRCMD_R(s)`, `SRCMD_W(s)`, and `SRCMD_X(s)`, as well as `SRCMD_RH(s)`, `SRCMD_WH(s)`, and `SRCMD_XH(s)` if applicable. RRID *s* Read Permission, RRID *s* Write Permission, and RRID *s* Instruction Fetch Permission describe the details of these registers.

Register `SRCMD_R(s)`, `SRCMD_W(s)`, and `SRCMD_X(s)` each has a single field, `SRCMD_R(s).md`, `SRCMD_W(s).md`, and `SRCMD_X(s).md`, respectively representing the read, write, and instruction fetch permissions of memory domain 0 to 30 for requester *s*. Register `SRCMD_RH(s)`, `SRCMD_WH(s)`, `SRCMD_XH(s)` each has a single field, `SRCMD_RH(s).mdh`, `SRCMD_WH(s).mdh`, and `SRCMD_XH(s).mdh`, respectively representing the read, write, and instruction fetch permissions of memory domain 31 to 62 for requester *s*. Register `SRCMD_RH(s)`,

`SRCMD_WH(s)`, and `SRCMD_XH(s)` are implemented if `HWCFG0.md_num` > 31.

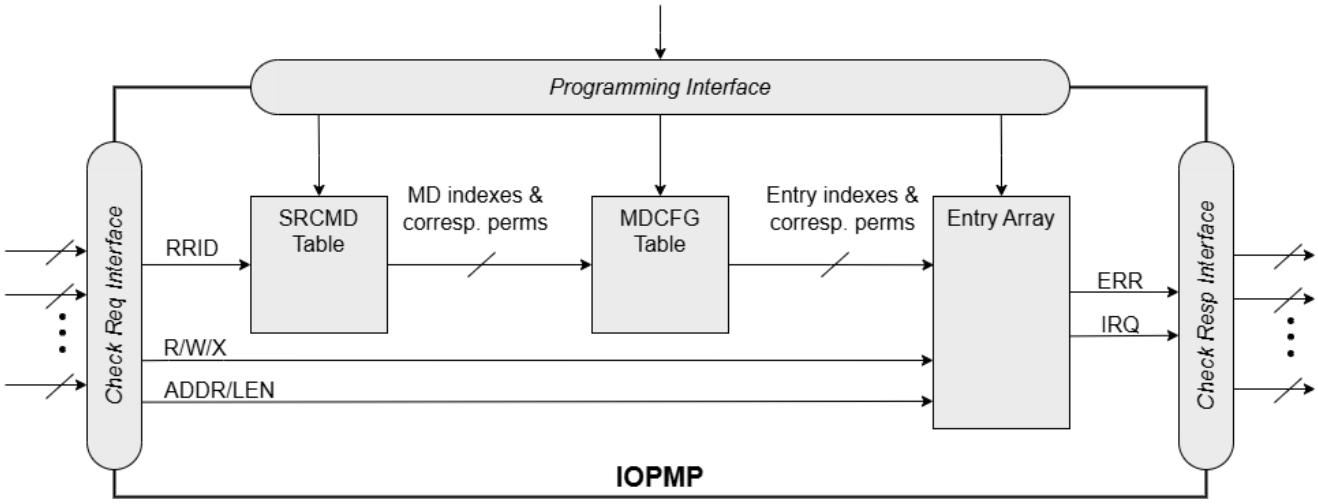

Figure 4. An example block diagram of an IOPMP with the SPS extension. It illustrates the checking flow of an IOPMP with the SPS extension. The IOPMP first looks up the SRCMD Table according to the RRID carried by the incoming transaction to retrieve associated MD indexes and the corresponding permissions related to these MDs. By the MD indexes, the IOPMP looks up the MDCFG Table to get the belonging entry indexes. The final step checks the access right according to the above entry indexes and corresponding permissions.

The extension shares the same locks as `SRCMD_EN(s)` and `SRCMD_ENH(s)`. Setting lock to `SRCMD_EN(s).l` locks `SRCMD_R(s)`, `SRCMD_RH(s)`, `SRCMD_W(s)`, `SRCMD_WH(s)`, `SRCMD_X(s)`, and `SRCMD_XH(s)`. `MDLCK.md[m]` locks `SRCMD_R(s).md[m]`, `SRCMD_W(s).md[m]`, and `SRCMD_X(s).md[m]` for all existing RRID *s*. `MDLCKH.mdh[m]` also locks `SRCMD_RH(s).mdh[m]`, `SRCMD_WH(s).mdh[m]`, and `SRCMD_XH(s).mdh[m]` for all existing RRID *s*. `SRCMD_R(s)`, `SRCMD_RH(s)`, `SRCMD_W(s)`, `SRCMD_WH(s)`, `SRCMD_X(s)`, and `SRCMD_XH(s)` can have prelocked bits fully or partially based on presets of `MDLCK.md`, `MDLCK.mdh`, and `SRCMD_EN.l`.

The extension has two sets of permission settings: one from IOPMP entry and the other from SPS registers (`SRCMD_R`/`SRCMD_W`/`SRCMD_X`/`SRCMD_RH`/`SRCMD_WH`/`SRCMD_XH`). A transaction is legal only if it is permitted by both the SRCMD Table (SPS) permission and the matching entry's permission. A transaction fails the SPS check if it violates either of the permission settings. That is, the extension can only be used to restrict the permissions defined on an entry, not grant new permissions.

5.3. Non-priority IOPMP entries

The baseline IOPMP specification defines a priority-based matching logic, where entries with lower indices are assigned a higher priority and take precedence. While this model provides deterministic behavior and robust security, it presents significant implementation challenges, particularly for systems requiring a large number of entries.

This priority-based model has two primary scalability limitations:

1. **Latency Impact**: As the number of priority entries increases, the complexity of the matching logic grows. An implementation must check a transaction against potentially many rules, ensuring that only the highest-priority match takes effect. This deep combinatorial logic can extend the critical path, forcing a longer clock period or requiring the insertion of additional pipeline stages, both of which increase the overall transaction check latency.
2. **Caching and Area Inefficiency**: Priority entries cannot be cached efficiently for high-speed lookups. A simple, parallel address match is insufficient, as the permission from a lower-priority match (even if it hits in a cache) can be subsequently overwritten by any higher-priority entry. This forces implementations to perform complex, sequential checks rather than simple, parallel ones, hindering area optimization in high-performance designs.

This extension introduces **non-priority entries** to address these scalability limitations, enabling reduced latency, power consumption, and area. Non-priority entries are all treated equally at the lowest priority, which allows their matching logic to be effectively cached, parallelized, and optimized, thus saving address matching circuitry.

This creates a clear trade-off:

- **Priority Entries** offer superior security. A small number of high-priority, locked entries can provide defense-in-depth, safeguarding critical regions even if runtime secure software is compromised.
- **Non-Priority Entries** offer superior performance, lower cost, and scalability for managing a large number of rules.

For systems requiring a large number of entries, a hybrid model using both types is recommended. This allows an implementation to achieve a balance between security and scalability, using priority entries for critical, static rules, and non-priority entries for the bulk of dynamic, performance-sensitive access control.

5.3.1. Functional Description

`HWCFG2.non_prio_en` indicates whether the IOPMP supports non-priority entries. When the extension is enabled, `HWCFG2.non_prio_en` is 1 and entries with indices greater than or equal to `HWCFG2.prio_entry` are non-prioritized. These are treated equally and assigned the lowest priority. Entries with indices less than `prio_entry` are prioritized. The value of `prio_entry` is implementation-dependent. Additionally, a field `HWCFG2.prio_ent_prog` indicates whether `prio_entry` is programmable. Hardware Configuration 2 describes the details of the fields.

Priority entries safeguard the most sensitive data, even in the event of secure software being compromised. However, implementing a large number of priority entries results in higher latency and increased area usage. In contrast,non-priority entries are treated equally and can be cached in smaller numbers. A mix of entry types provides flexibility in balancing security and performance.

An entry qualifies as a matching entry with the extension for an incoming transaction if:

- For priority entries, its region covers any byte of the transaction,
- For non-priority entries, its region covers all bytes of the transaction,
- It is associated with the RRID carried by the transaction; and
- It holds the highest priority among entries that meet the previous criteria.

Multiple matching entries are allowed for non-priority entries because they share the lowest priority.

IOPMPs don't report error type = 0x04 "Partial hit on a priority rule" on non-priority entries because non-priority entries must cover all bytes of the transaction.

If one matching entry is priority, the reaction is the same as baseline IOPMP entry array.

5.3.2. Error Reporting

If one matching entry is non-priority, the transaction is legal if any matching entry permits its access type. If no matching entry permits, the transaction is illegal with error type = "illegal read access" (0x01) for read access transaction, "illegal write access/AMO" (0x02) for write access/AMO transaction, or "illegal instruction fetch" (0x03) for instruction fetch transaction.

For an illegal transaction matching multiple non-priority entries, if the interrupt is triggered or the bus error response is returned, `ERR_REQID.eid` stores the index of any of them.

5.4. Per-entry Interrupt/Bus Error Suppression

This extension provides a fine-grained, per-entry mechanism to selectively suppress interrupt triggers and bus error responses for specific access violations.

In modern high-performance systems, I/O agents may employ aggressive speculative prefetching to improve performance. While beneficial, these prefetchers may speculatively access memory locations just outside their intended operational regions, such as "guard regions" or adjacent memory boundaries.

If these guard regions are protected by an IOPMP entry (which is a correct and secure configuration), the speculative prefetch would be correctly identified as a violation. However, this "benign" violation can lead to a flood of unnecessary interrupt alarms and bus errors, creating significant system noise and increasing the burden on the secure monitor.

The per-entry suppression extension addresses this problem. It allows software to define these known guard regions with specific IOPMP entries and then set flags (e.g., `sire`, `sere`) to suppress the interrupt and bus error for violations at that specific entry. This ensures that speculative prefetches do not stall the system or generate false alarms, while still preventing access to truly illegal content and without adding interference to the system.

The per-entry suppression extension only extends the error reactions, which are triggered when an entry is matched but does not grant transaction permission to operate.

5.4.1. Interrupt Suppression

The bit `peis` in register `HWCFG2` (Hardware Configuration 2) indicates whether the IOPMP supports per-entry interrupt suppression. If `HWCFG2.peis` is 1, every entry *i* has three interrupt suppression bits, `sire`, `siwe`, `sixe`, in register `ENTRY_CFG(i)` (Entry *i* Configuration with All Extensions). The bits suppress interrupt triggering due to illegal read access, illegal writes access/AMO and illegal instruction fetch respectively.

When an illegal transaction occurs with error type "illegal read access" (0x01), "illegal write access/AMO" (0x02), or "illegal instruction fetch" (0x03), an interrupt is triggered if the global interrupt is enabled (`ERR_CFG.ie`) and not suppressed by `sire`, `siwe`, or `sixe`. Considering Entry *i* matches an illegal transaction, the condition for the interrupt for each type of illegal access can be described as follows:

- Illegal read access (0x01):
  `ERR_CFG.ie && !ENTRY_CFG(i).sire`
- Illegal write access/AMO (0x02):
  `ERR_CFG.ie && !ENTRY_CFG(i).siwe`
- Illegal instruction fetch (0x03):
  `ERR_CFG.ie && !ENTRY_CFG(i).sixe`

Where `!` represents logical NOT, and `&&` represents logical AND.

5.4.2. Bus Error Suppression

The bit `pees` in register `HWCFG2` (Hardware Configuration 2) indicates whether the IOPMP supports per-entry bus error suppression. If `HWCFG2.pees` is 1, every entry *i* has three interrupt suppression bits, `sere`,

`sewe`, `sexe`, in register `ENTRY_CFG(i)` (Entry *i* Configuration with All Extensions). The bits suppress bus error due to illegal read access, illegal writes access/AMO and illegal instruction fetch respectively.

When an illegal transaction occurs with error type "illegal read access" (0x01), "illegal write access/AMO" (0x02), or "illegal instruction fetch" (0x03), an IOPMP responds bus error if the bus error is not suppressed by `ERR_CFG.rs` and corresponding per-entry suppression bit. `rs` globally suppresses returning a bus error on illegal access. When global suppression is disabled, individual per-entry suppression is possible using `sere`, `sewe`, and `sexe` in `ENTRY_CFG(i)` for illegal read access, illegal write access/AMO, and illegal instruction fetch, respectively. Considering Entry *i* matches an illegal transaction, the condition for a bus error response for each access type can be described as follows:

- Illegal read access (0x01):
  `!ERR_CFG.rs && !ENTRY_CFG(i).sere`
- Illegal write access/AMO (0x02):
  `!ERR_CFG.rs && !ENTRY_CFG(i).sewe`
- Illegal instruction fetch (0x03):
  `!ERR_CFG.rs && !ENTRY_CFG(i).sexe`

Where `!` represents logical NOT, and `&&` represents logical AND.

5.4.3. Per-entry Suppression on Non-priority Entries

An IOPMP may match multiple entries when it supports Section 5.3, "Non-priority IOPMP entries". To suppress interrupt, all matching entries must suppress interrupt together. Similarly, to suppress bus error response, all matching entries also must suppress bus error response together. The following describes the detailed relation.

For the cases with multiple matched non-priority entries indexed by i_0, i_1, ..., i_N , the condition of per-entry interrupt suppression is:

- Illegal read access (0x01):
  `ERR_CFG.ie && ( !ENTRY_CFG(`i_0`).sire || !ENTRY_CFG(`i_1`).sire || … || !ENTRY_CFG(`i_N`).sire )`
- Illegal write access/AMO (0x02):
  `ERR_CFG.ie && ( !ENTRY_CFG(`i_0`).siwe || !ENTRY_CFG(`i_1`).siwe || … || !ENTRY_CFG(`i_N`).siwe )`
- Illegal instruction fetch (0x03):
  `ERR_CFG.ie && ( !ENTRY_CFG(`i_0`).sixe || !ENTRY_CFG(`i_1`).sixe || … || !ENTRY_CFG(`i_N`).sixe )`

The condition of per-entry bus error response suppression is:

- Illegal read access (0x01):
  `!ERR_CFG.rs && ( !ENTRY_CFG(`i_0`).sere || !ENTRY_CFG(`i_1`).sere || … || !ENTRY_CFG(`i_N`).sere )`
- Illegal write access/AMO (0x02):
  `!ERR_CFG.rs && ( !ENTRY_CFG(`i_0`).sewe || !ENTRY_CFG(`i_1`).sewe || … || !ENTRY_CFG(`i_N`).sewe )`
- Illegal instruction fetch (0x03):
  `!ERR_CFG.rs && ( !ENTRY_CFG(`i_0`).sexe || !ENTRY_CFG(`i_1`).sexe || … || !ENTRY_CFG(`i_N`).sexe )`

Where `!` represents logical NOT, `||` represents logical OR, and `&&` represents logical AND.

5.5. Multi-Faults Record

The primary error capture registers are designed to lock upon detecting the **first** violation to preserve its

details. This behavior, while necessary for root cause analysis, creates a "blind spot". Once `ERR_INFO.v` is set, all **subsequent** violations from any RRID are ignored, and their occurrence is permanently lost. The secure monitor remains unaware of these additional faults until it handles the interrupt and clears the `ERR_INFO.v` bit, at which point it's too late to know what else has failed.

This extension, Multi-Faults Record (MFR), addresses this issue by providing a low-cost, secondary logging mechanism. It does not record the full details (like address or type) of subsequent faults. Instead, it maintains a simple, high-performance bitmap to log **which RRIDs** have generated at least one subsequent violation. This allows the secure monitor to efficiently identify all misbehaving agents during a single fault event, rather than only discovering the first one.

The extension maintains a bit, referred to as SV[*s*], for each RRID *s*. When one or more subsequent violations are issued from an RRID, the corresponding bit is set. To retrieve these SVs, a 32-bit register `ERR_MFR` is used.

To retrieve these SVs, a 32-bit register `ERR_MFR` (Multi-Faults Record) is used. Every 16 contiguous SVs are grouped together into a record window, which is indexed by a 12-bit field, `svi`. When `ERR_MFR` is read, the IOPMP sequentially scans all record windows towards the last window from original position (`svi`) until a violation is found. Once the last available record window is scanned, the next record window to be scanned is the first record window (i.e., SV[0] ~ SV[15]). The original position can be set by writing an index of record window to `svi`. If found, the status bit `svs` is set, and `svi` indexes the window containing the first found set SV. The 16-bit field `svw` reflects the record window indexed by `svi`, where `svw[j]`=SV[`svi` * 16 + *j*]. After the register is read out, all bits in the record window are cleared. If not found, `svs` and `svw` return zeros and `svi` keeps the same. Moreover, the bit `svc` in the `ERR_INFO` (Error Information with All Extensions) indicates if any subsequent violation is in the log.

The read only bit `mfr_en` in register `HWCFG2` (Hardware Configuration 2) indicates if the IOPMP implements the extension.

5.6. Message-Signaled Interrupts (MSI) Extension

The extension can trigger message-signaled interrupts (MSI) by writing a word of data to a specific physical address. It can be used in the systems built with an Incoming Message-Signaled Interrupt Controller (IMSIC) [3].

The bit `msi_en` in register `HWCFG2` (Hardware Configuration 2) indicates whether the IOPMP supports MSI extension.

The address is MSI address and is specified by `ERR_MSIADDR` and `ERR_MSIADDRH` (MSI Message Address), while the content to write is stored in the field `msidata` in the register `ERR_CFG` (Error Configuration with All Extensions). The `ERR_MSIADDR`, `ERR_MSIADDRH`, and `ERR_CFG` are locked by the `ERR_CFG.l`.

`ERR_MSIADDRH` is available if the target address of IMSIC uses over 34 bits (greater than 0x2_0000_0000). `ERR_MSIADDR` encodes bit 31 to bit 0 of the address, while `ERR_MSIADDRH` encodes bit 63 to bit 32. When the target address of IMSIC uses less than or equal to 34 bits, `ERR_MSIADDR` encodes bit 33 to bit 2. `ERR_MSIADDRH` is available only when `HWCFG0.addrh_en` = 1 and `HWCFG2.msi_en` = 1.

The bit `msi_sel` in register `ERR_CFG` (Error Configuration with All Extensions) indicates whether the IOPMP triggers interrupt by MSI or wired interrupt. IOPMP triggers MSI when `ERR_CFG.msi_sel` = 1 and triggers wired interrupt when `ERR_CFG.msi_sel` = 0. `ERR_CFG.msi_sel` can be programmable or hardwired.

The bit `msi_werr` in register `ERR_INFO` (Error Information with All Extensions) indicates whether a write access to trigger an IOPMP-originated MSI has failed. It is asserted when the write access to trigger an IOPMP-originated MSI has failed. When it's not available, it must be zero. Writing 1 to `ERR_INFO.msi_werr`

clears the bit.

5.7. Safe Runtime Configuration

This section introduces an extension to stall transactions by RRIDs and describes how to use the extension to update IOPMP's settings safely in runtime.

At times, attempting to configuring all IOPMP settings when the system starts can be difficult or even impossible, especially before I/O agents are active or connected to the system. As a result, it is necessary to update IOPMP settings during runtime.

To configure all IOPMP settings during runtime, security software without the extension can configure additional MDs for different settings in MDCFG Table and entry array, and switch the associated MDs for an RRID in SRCMD Table from a single access.

The aforementioned method is not applicable if an IOPMP supports number of MD greater than 31 and its control port only supports 32-bit access. Security software needs two accesses to switch the association in the configuration. It results in an incomplete setting between the first access and the second access and potentially create a vulnerability.

However, the aforementioned method may not be suitable for certain systems. Another method to update IOPMP settings is necessary. This may occur when a device is enabled, disabled, added, removed, or when the accessible area of a device changes. When updating, it is important to avoid checking transactions in an incomplete setting. However, updating IOPMP settings often involves a series of control accesses, and if a transaction check occurs during the update, it can potentially create a vulnerability. It could be a heavy burden for the security software to guarantee that no transactions are in progress from all related requesters. This extension enables the method for updating IOPMP's settings that avoids checking transactions in an incomplete setting and burdening the security software.

5.7.1. Atomicity Requirement

The Atomicity Requirement is that when updating an IOPMP, all transactions from receiver ports must be checked either before any changes are made or after all changes are completed. Essentially, checking a transaction in a partial setting should be avoided. The succeeding sections will describe the mechanism that helps the security software satisfy this requirement.

5.7.2. Programming Steps

The general approach for a security software to the atomicity requirement has three major steps, conceptually described as follows:

- Step 1: Stall the related transactions that the following updates may impact.
- Step 2: Update IOPMP's settings.
- Step 3: Resume the transactions stalled in Step 1.

Stalling a transaction means the IOPMP stops checking its permission until it is resumed. In the meantime, the transaction should wait. For Step 1, it might be necessary to verify if the stalling has occurred since some implementations could not immediately stall the desired transactions from receiver ports. Following this, execute the IOPMP updates as Step 2, and finally, resume all stalled transactions in Step 3.

In some cases, Step 1 and Step 3 may be skipped as long as no transaction check is performed

within Step 2. Writing the SRCMD of an RRID is an example because it involves a single write.

Stalling a transaction could be implemented in the following ways but not limited to: (1) IOPMP may enqueue the transactions to stall but not be checked, (2) ask the requester to retry the transaction by a retry message, and (3) enqueue first and ask to retry if the queue is full.

5.7.3. Stall Transactions

For Step 1, it's possible to postpone all transactions until all updates are finished. However, this could cause unrelated transactions to experience unnecessary delays. This might not be tolerable for devices that require low latency, like a display controller that periodically retrieves a frame from its video buffer. This section explains the mechanism that only stalls specific transactions to prevent the aforementioned scenario and ensure the atomicity requirement. All the features mentioned below are optional.

In Step 2, the decision of whether a transaction should be stalled cannot reference the current settings because these settings are under updating. Therefore, the only reliable information for this decision is the RRID carried by the transaction. To simplify the following descriptions, we use *rrid_stall* to represent an abstract bit array, where *rrid_stall*[*s*] = 1 indicates that transactions with RRID *s* must be stalled. The IOPMP stalls a transaction based on its corresponding *rrid_stall*. Please note that it may not be an actual bit array in practice just for easier expression and is not directly accessible by software.

Register `MDSTALL` and `MDSTALLH` (MD-Based Stall Control) control *rrid_stall*. `MDSTALL.md` and `MDSTALLH.mdh` are used to select MDs. Denote *stall_by_md* to represent the concatenation of `MDSTALL.mdh` and `MDSTALL.md`. That is, *stall_by_md*[30:0] is `MDSTALL.md[30:0]`, and *stall_by_md*[62:31] is `MDSTALLH.mdh[31:0]` if any. An MD *m* is selected when *stall_by_md*[*m*] = 1. `MDSTALL.exempt` controls the polarity of the selection. An IOPMP updates *rrid_stall* only when `MDSTALL.exempt` is written. When `MDSTALL.exempt` is written to 0, *rrid_stall*[*s*] is set if RRID *s* is associated with a selected MD *m* (*stall_by_md*[*m*] = 1). The SRCMD Table defines the association of RRID *s* and MD *m*. Conversely, when `MDSTALL.exempt` is written to 1, *rrid_stall*[*s*] must be set if RRID *s* is not associated with any selected MD. This relation can be formulated as follows:

```
rrid_stall[s] ⇐ MDSTALL.exempt ^ ( Reduction_OR (SRCMD(s).md & stall_by_md));
```

Where:

- `^` is bitwise logical exclusive OR operator,
- `&` is bitwise logical AND operator, and
- `Reduction_OR()` is a function that applies bitwise logical OR across all input values.

For any unimplemented MD, the corresponding bit in `MDSTALL.md` or `MDSTALLH.mdh` should be wired to 0.

rrid_stall should be updated only when `MDSTALL.exempt` is written, that is, when `MDSTALL` is written. When `MDSTALLH` is written, the only action is to hold the value.

Although rrid_stall is derived from the SRCMD Table, it should be updated only when `MDSTALL.exempt` is written. We cannot use the table when stalling transactions because the table updates may still be in progress when the IOPMP decides if a transaction should be stalled. Writing `MDSTALL` is just like capturing a snapshot of the table at the moment right before updating any settings. The rrid_stall reflects the SRCMD Table before any updates.

When writing `MDSTALL`, the specification only defines the transactions with RRID=s must be stalled for all rrid_stall[s] = 1. It doesn't request the rest of the transactions to stall or not. It only asks that all transactions be resumed when writing `MDSTALL` with a zero.

5.7.4. Cherry Pick

Writing `RRIDSCP` register (RRID Cherry Pick Stall Control) is an optional substep after writing `MDSTALL` within Step 1 of Programming Steps. Software can select or deselect the transactions with specific RRIDs to stall if the previous writing `MDSTALL`/`MDSTALLH` doesn't stall all the desired transactions. Software can write the register zero or multiple times in the substep.

`MDSTALL`/`MDSTALLH` only stall the RRIDs based on the SRCMD Table at the moment of writing `MDSTALL`. rrid_stall is not updated when the SRCMD Table is updated. Besides, rrid_stall is set or clear only for those RRIDs associated with given MDs. The RRIDSCP is used to fine-tune the rrid_stall if writing `MDSTALL`/`MDSTALLH` doesn't fulfill the desired rrid_stall values. Software should, whenever possible, prioritize using the `MDSTALL`/`MDSTALLH` batch mechanism to manage the rrid_stall state. While RRIDSCP offers granular control, it introduces significant software complexity. Its use is primarily intended for complex scenarios where the MD-based grouping logic of `MDSTALL` is insufficient to precisely define the set of RRIDs that must be stalled or exempted.

The `RRIDSCP` register comprises two fields: a 2-bit `RRIDSCP.op` and a field for `RRIDSCP.rrid`. By setting `RRIDSCP.op` = 1, the *rrid_stall*[*s*] is set for *s* = `RRIDSCP.rrid`. Conversely, by setting `RRIDSCP.op` = 2, the *rrid_stall*[*s*] is clear for *s* = `RRIDSCP.rrid`. This register is used to fine-tune the result of writing `MDSTALL`/`MDSTALLH`. The writing `RRIDSCP.op` = 0 queries the value of *rrid_stall*[*s*] for a valid *s*. The value will be returned in the two most significant bits in the following reading `RRIDSCP`. `RRIDSCP.op`=3 is reserved.

5.7.5. Faulting stalled transactions

Faulting stalled transactions can be allowed. To allow faulting over stalling when an IOPMP cannot accommodate more stalled transactions, such as when the receiver port lacks a buffer for transaction storage, set bit `stall_violation_en` in register `ERR_CFG` (Error Configuration with All Extensions) to 1 before Step 1 of Programming Steps. If any transaction is faulted due to the stalled transactions, the error information shall be logged in `ERR_INFO` (Error Information with All Extensions), where `ERR_INFO.etype` = 0x07 (error due to stalled transactions). The procedure for faulting checking transactions is identical to Steps 1-3 of Programming Steps, except that, besides stalling and delaying the transactions, transactions can be faulted and cannot be resumed if the IOPMP can't handle more stalled transactions.

In certain implementations, rather than stalling the related transactions, the system may opt to fault the checking transactions during an IOPMP atomic update. Faulting transactions can be advantageous if the system lacks sufficient buffer capacity to record and store all transactions during the IOPMP programming process.

This function also can prevent the risk of deadlock in some systems. If an implementation or a system doesn't have sufficient buffer capacity for handling all stalled transactions during programming, the stalled transactions may backpressure from receiver port of the IOPMP and then cause a hang on the port due to its implementation limitation. Therefore, the hang on the port potentially causes a deadlock in the system since transactions for programming IOPMP during Step 2 and Step 3 of Programming Steps have the possibility to hang indirectly in the circumstance.

5.7.6. Resume Stall

In order to resume all stalled transactions, writing 0 to `MDSTALLH`, then writing 0 to `MDSTALL`. This corresponds to Step 3 of Programming Steps. After `MDSTALL` is written by zero, an IOPMP should clear `MDSTALL.is_busy` within some time, at which point all transactions have been resumed.

5.7.7. The Order to Stall

In Step 1 of Programming Steps, `MDSTALL` can be written at most once and before a resume. Writing a non-zero value to `MDSTALL` multiple times after a resume leads to RRIDs' stall states being undefined.

After `MDSTALL` and all `RRIDSCP` are written, the action to stall desired transactions may not take effect immediately in some implementations in which the subsequent setting updates (Step 2) could affect the transactions still under checking. To determine whether the action takes effect, software can read back and check the bit `MDSTALL.is_busy`, which is in the same location as `MDSTALL.exempt` on a write. `is_busy` = 0 indicates it has taken effect or no previous write; otherwise, it has not. A new writing `RRIDSCP` may temporarily switch `is_busy` to 1 and then switch to 0 at some time. `is_busy` can be wired to 0 if any `MDSTALL` and `RRIDSCP` writing won't cause a race condition of the transactions still under checking and the subsequent setting updates.

Based on Programming Steps, complete steps to program an IOPMP should be followed.

- Step 1.1: write `MDSTALLH`
- Step 1.2: write `MDSTALL` once // exactly once
- Step 1.3: write `RRIDSCP` zero or more times
- Step 1.4: poll until `MDSTALL.is_busy` == 0 // to ensure all stalls takes effect if necessary for the implementation
- Step 2: update IOPMP's settings
- Step 3.1: write 0 to `MDSTALLH`
- Step 3.2: write 0 to `MDSTALL` // resume all transactions
- Step 3.3: poll until `MDSTALL.is_busy` == 0 // optional, to ensure all resumes take effect.

Some steps may be skipped according to the actual implementation.

5.7.8. Implementation Options

`HWCFG2.stall_en` indicates if the IOPMP implements the extension, which are `MDSTALL`, `MDSTALLH`, `RRIDSCP`, `ERR_CFG.stall_violation_en`, and additional error type = 0x07 "error due to a stalled transaction".

All registers and fields described in Section 5.7, "Safe Runtime Configuration" (`MDSTALL`, `MDSTALLH`, `RRIDSCP`, and `ERR_CFG.stall_violation_en`) are optional. Moreover, these features could be partially implemented.

In `MDSTALL.md` and `MDSTALLH.mdh`, not every bit should be implemented even though the corresponding MD is implemented. An unimplemented bit means unselectable and should be wired to zero. To test which bits are implemented, software can write all 1's to `MDSTALL.md` and `MDSTALLH.mdh` and then read them back. An implemented bit returns 1.

If an IOPMP implementation has fewer than 32 memory domains, `MDSTALLH` should be wired to zero.

An example of partial implementation of `MDSTALL.md`/`MDSTALLH.mdh` *is a system with a display controller, which is a latency-sensitive device. On updating the IOPMP, the transactions initiated from the display controller should not be stalled. Thus, software can always use* `MDSTALL.exempt`*=1 and* `MDSTALL.md[j]`*=1, where MD j is the memory domain for the frame buffer that the display controller keeps accessing. Thus, the system only needs to implement* `MDSTALL.md[j]`.

`MDSTALL.is_busy` can be wired to 0 if any `MDSTALL` and `RRIDSCP` writing won't cause a race condition of the transactions still under checking and the subsequent setting updates.

If whole `MDSTALL` is not implemented, `MDSTALL`, `MDSTALLH`, `RRIDSCP` and `ERR_CFG.stall_violation_en` should always return zero.

If `RRIDSCP` is not implemented, it always returns zero. Software can test if it is implemented by writing a zero and then reading it back. Any IOPMP implementing `RRIDSCP` should not return a zero in `RRIDSCP.stat` in this case.

It is unnecessary to allow every implemented RRID to be selectable by `RRIDSCP.rrid`. If an unimplemented or unselectable RRID is written into `RRIDSCP.rrid`, it returns `RRIDSCP.stat` = 3.

`ERR_CFG.stall_violation_en` is a WARL field so it can be programmable or fixed.

Bibliography

[1] “The RISC-V Instruction Set Manual Volume II: Privileged Architecture.” [Online]. Available: github.com/riscv/riscv-isa-manual.

[2] “RISC-V IOMMU Architecture Specification.” [Online]. Available: github.com/riscv-non-isa/riscv-iommu.

[3] “RISC-V Advanced Interrupt Architecture.” [Online]. Available: github.com/riscv/riscv-aia.

A. Application Note

Introduction

This application note provides practical guidelines for implementing and configuring the IOPMP in various system scenarios. The primary objectives of these guidelines are to enable area-efficient implementations, simplify design complexity, and optimize IOPMP configurations for specific use cases.

The IOPMP specification defines a comprehensive protection mechanism with extensive configurability. However, not all systems require the full feature set. This application note demonstrates how to leverage optional features and configuration strategies to achieve:

- **Area reduction**: Minimize hardware resource utilization by implementing only necessary features
- **Design simplification**: Reduce implementation complexity for specific use cases
- **Configuration optimization**: Streamline IOPMP setup and programming for common scenarios
- **Performance enhancement**: Optimize rule checking and table lookup operations

Each section addresses a specific implementation strategy or use case, providing both the technical mechanisms and practical examples. These guidelines are particularly valuable for embedded systems, resource-constrained environments, and specialized applications where standard configurations may be over-provisioned.

The techniques described in this application note are optional and implementation-specific. System designers should evaluate their requirements and select appropriate strategies based on their specific constraints and objectives.

Registers

This section summarizes the additional registers and fields required when implementing any of the application variants. Detailed register definitions are provided in their respective sections.

Table 8. Register and field summary

Offset	Register	Field	Bits	Related section(s)
0x0014	`HWCFG3`	`mdcfg_fmt`	1:0	Section A.6, “MDCFG Table Reduction”
		`srcmd_fmt`	3:2	Section A.4, “SRCMD Table Reduction”
		`md_entry_num`	10:4	Section A.6, “MDCFG Table Reduction”
		`xinr`	11:11	Section A.3, “Data Only Devices and Buses”
		`no_x`	12:12	Section A.2, “Instruction Fetch Protection Devices”
		`no_w`	13:13	Section A.1, “Write Protection Devices”
		`rrid_transl_en`	14:14	Section A.7.2, “Cascading IOPMP”
		`rrid_transl_prog`	15:15	
		`rrid_transl`	31:16	
0x1000 + (m) × 32	`SRCMD_PERM(`*m*`)`	All		Section A.4.3, “SRCMD Table in the MD-indexed Format”
0x1004 + (m) × 32	`SRCMD_PERMH(`*m*`)`	All		

Hardware Configuration 3 (`HWCFG3`)

Field	Bits	R/W	Default	Description
`mdcfg_fmt`	1:0	R	IMP	Indicate the MDCFG format: • 0x0: Format 0. MDCFG Table is implemented. • 0x1: Format 1. No MDCFG Table. `HWCFG3.md_entry_num` is fixed. • 0x2: Format 2. No MDCFG Table. `HWCFG3.md_entry_num` is programmable. • 0x3: reserved.
`srcmd_fmt`	3:2	R	IMP	Indicate the SRCMD Table format: • 0x0: Format 0 (baseline). `SRCMD_EN(s)` and `SRCMD_ENH(s)` are available. • 0x1: Exclusive Format. No SRCMD Table. RRID *i* associates exclusively with memory domain *i*, *i* = 0 … `HWCFG0.md_num`-1. • 0x2: MD-indexed Format. `SRCMD_PERM(m)` and `SRCMD_PERMH(m)` are available. • 0x3: reserved.
`md_entry_num`	10:4	WARL	IMP	When `HWCFG3.mdcfg_fmt` = • 0x0: must be zero • 0x1 or 0x2: `md_entry_num` indicates each memory domain has exactly (`md_entry_num` + 1) entries `md_entry_num` is locked if `HWCFG0.enable` is 1.
`xinr`	11:11	R	IMP	Indicates whether the IOPMP treats the instruction fetch accesses as data read accesses. All fields related to instruction fetch can be fixed to 0, or unavailable.
`no_x`	12:12	R	IMP	When `no_x`=1, the IOPMP denies all instruction fetch transactions; otherwise, it depends on the `x`-bit in `ENTRY_CFG(i)`.
`no_w`	13:13	R	IMP	Indicate if the IOPMP always denies write accesses as if no rule matched.
`rrid_transl_en`	14:14	R	IMP	Indicate if tagging a new RRID on the requester port is supported.
`rrid_transl_prog`	15:15	W1CS	IMP	A write-1-clear bit that is sticky to 0. Indicates if the `rrid_transl` field is programmable. Supported only for `rrid_transl_en`=1, otherwise, wired to 0.
`rrid_transl`	31:16	WARL	IMP	The RRID tagged to outgoing transactions. Supported only for `rrid_transl_en`=1. It is writable only when `rrid_transl_prog`=1.

MD *m* RRID Permission (`SRCMD_PERM(m)` and `SRCMD_PERMH(m)`)

`SRCMD_PERM(m)` is read and write permissions of RRID 0 ~ 15 for MD *m*.

Field	Bits	R/W	Default	Description
`perm`	31:0	WARL	DC	Holds two bits per RRID that give the RRID's read and write permissions for memory domain *m*. Bit 2**s* holds the read permission for RRID *s*, and bit 2**s*+1 holds the write permission for RRID *s*, where *s*≤15. The instruction fetch permission is considered the same as the read permission.

`SRCMD_PERMH(m)` is read and write permissions of RRID 16 ~ 31 for MD *m*.

Field	Bits	R/W	Default	Description
`permh`	31:0	WARL	DC	Holds two bits per RRID that give the RRID's read and write permissions for memory domain *m*. Bit 2*(*s*-16) holds the read permission for RRID *s*, and bit 2*(*s*-16)+1 holds the write permission for RRID *s*, where *s*≥16. The register is implemented when `HWCFG1.rrid_num` > 16. The instruction fetch permission is considered the same as the read permission.

A.1. Write Protection Devices

Systems that are primarily used as read-only memory regions, such as Flash memory or read-only memory (ROM), can leverage global write protection to reduce implementation complexity and hardware overhead. The IOPMP provides the `no_w` configuration bit in the `HWCFG3` register (Hardware Configuration 3) to enable this functionality.

When `no_w` is set to 1, the IOPMP denies all write transactions regardless of entry rule configurations, reporting them with error type "not hit any rule" (0x05). This global configuration strategy eliminates the need for per-entry write permission checking logic and removes the requirement to configure write permission bits in individual entry rules, thereby reducing hardware resources and implementation complexity.

The `no_w` bit is a read-only implementation parameter that indicates whether the IOPMP instance supports this global write protection feature. System designers can leverage this configuration for applications where write access control is unnecessary, achieving both area savings and design simplification.

A.2. Instruction Fetch Protection Devices

The `no_x` bit enables global instruction fetch protection, which is particularly useful for protecting data-only memory regions such as peripherals which don't have any instruction of processors. This feature simplifies implementations by reducing the need for programming individual permission bits and minimizing the overhead of per-entry rule lookups.

The global instruction fetch behavior is controlled by `no_x` bit in the `HWCFG3` register (Hardware Configuration 3):

- `no_x = 0`: The IOPMP performs instruction fetch permission checking based on entry-level rules
- `no_x = 1`: The IOPMP denies all instruction fetch transactions and reports them with error type "not hit any rule" (0x05)

A.3. Data Only Devices and Buses

IOPMPs may only protect from data only device such as general DMA. The IOPMP under the general DMA in Figure 5 illustrates an example integration. In this scenario, instruction fetch permission checking is not

required. Therefore, all fields related to instruction fetch can be fixed to 0, or not implemented. This can apply to `x` in `ENTRY_CFG`, `sixe`/`sexe` of Per-entry Interrupt/Bus Error Suppression in `ENTRY_CFG`, and `SRCMD_X`/`SRCMD_XH` of SPS extension.

For some small-scale systems which have a single RISC-V hart with PMP and data-only requesters, or for some system buses that don't provide any instruction fetch hint, the IOPMP can treat the instruction fetch accesses as data read accesses. This simplifies both the system and IOPMP configuration by fixing all instruction fetch permission bits to 0, or making them unavailable. The field `xinr` in register `HWCFG3` (Hardware Configuration 3) indicates whether the IOPMP treats the instruction fetch accesses as data read accesses.

A system with a single RISC-V hart that includes PMP and data-only requesters can enforce execution permission checking entirely within the hart using its PMP. Consequently, the system bus in such a configuration only needs to support general read and write accesses.

A.4. SRCMD Table Reduction

The IOPMP specification provides SRCMD Table reduction strategies to minimize hardware resource utilization in systems with simplified access control requirements. These strategies enable area-efficient implementations by reducing or eliminating SRCMD Table lookups while maintaining essential protection functionality.

A.4.1. Source-Enforcement

Systems with requesters that already implement firewall mechanisms, such as RISC-V harts with Physical Memory Protection (PMP), can optimize IOPMP implementation by applying rule checking selectively. This approach, known as source-enforcement (IOPMP-SE), enables significant area savings and design simplification when protection is needed only for a subset of requesters.

Source-enforcement is particularly beneficial in heterogeneous systems where some requesters possess their own protection mechanisms while others, such as DMA engines or specialized accelerators, require external protection. By configuring the IOPMP to enforce rules only for unprotected requesters, implementations can achieve substantial hardware resource reduction without compromising system security.

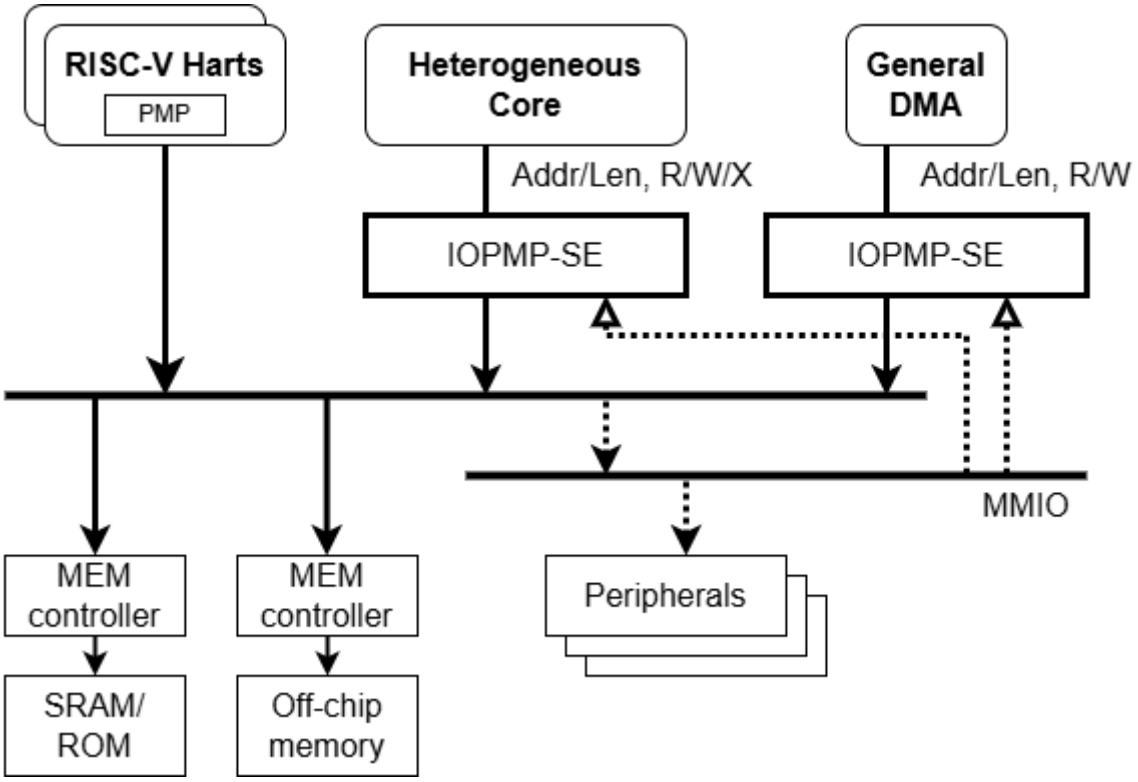

Figure 5. Example implementation of source-enforcement where RISC-V harts use PMP while heterogeneous cores and DMA rely on IOPMP-SE.

A.4.2. SRCMD Table in the Exclusive Format

Exclusive Format implements a direct mapping where RRID *i* associates exclusively with memory domain *i*. This one-to-one mapping eliminates the need for SRCMD Table lookups, providing benefits in area efficiency, access latency reduction, and implementation complexity. The format supports scenarios with a single requester or multiple requesters requiring identical access permissions.

Exclusive Format is particularly effective for embedded systems and resource-constrained environments where minimizing hardware overhead is critical. The format supports up to 63 RRIDs and eliminates the physical SRCMD Table implementation entirely. However, designers should note that shared memory regions require duplicated entry settings in this format, which may impact configuration flexibility in systems with extensive resource sharing requirements.

The SRCMD Table format is discovered through the `srcmd_fmt` field in `HWCFG3` register (Hardware Configuration 3). `HWCFG3.srcmd_fmt` = 1 indicates the SRCMD Table format is Exclusive Format.

A.4.3. SRCMD Table in the MD-indexed Format

This section introduces the MD-indexed Format (Format 2) for the SRCMD Table. This format is provided as an alternative implementation, driven by a key strategic goal: to lower the barrier for migrating existing, non-RISC-V systems to the RISC-V ecosystem.

The primary objective of non-ISA specifications like IOPMP is to accelerate the broad adoption of RISC-V CPUs. However, the baseline RRID-indexed programming model (Format 0), while powerful, may present a significant software migration obstacle for platforms with established, legacy programming models. Some existing systems architect their access control from the perspective of the resource (the memory regions) rather than the requester (the RRID).

The MD-indexed format directly supports this familiar, resource-centric paradigm. By providing a physical

SRCMD Table indexed by the Memory Domain (MD) instead of the RRID, it allows existing security software and established programming models to be ported with minimal modification.

MD-indexed Format implements a physical SRCMD Table array similar to the baseline SRCMD Table, but indexed by memory domain rather than RRID. For memory domain *m*, registers `SRCMD_PERM(m)` and `SRCMD_PERMH(m)` (MD *m* RRID Permission) are implemented at the same addresses as `SRCMD_EN(s)` and `SRCMD_ENH(s)` in the baseline format. This format allows per-RRID permission configuration for each memory domain, outputting permissions for every memory domain based on the requesting RRID. MD-indexed Format supports up to 32 RRIDs. The SPS extension is not supported in this format.

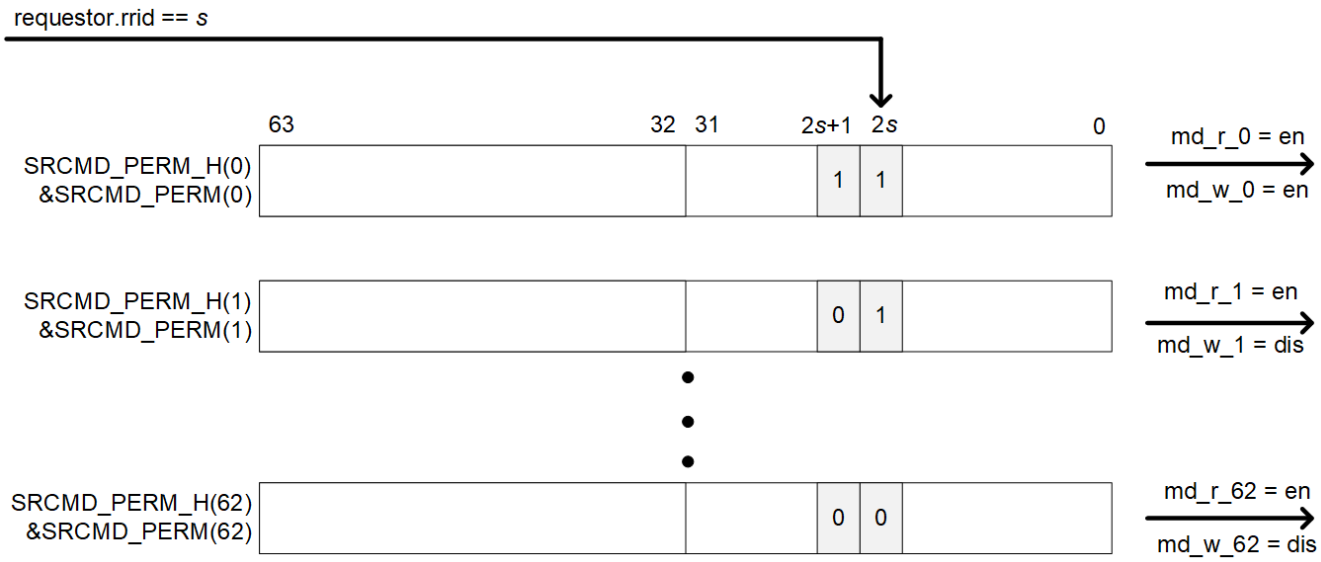

Figure 6. An example block diagram of SRCMD Table MD-indexed Format Implementation.

`SRCMD_PERM(m)` and `SRCMD_PERMH(m)` are available when `HWCFG3.srcmd_fmt` = 2. In MD-indexed Format, an IOPMP checks both the permission of `SRCMD_PERM(H)(m)` and the `ENTRY_CFG.r/w/x` permission. The instruction fetch permission of `SRCMD_PERM(H)(m)` is considered the same as its corresponding read permission. A transaction is legal if either of them allows the transaction.

A.5. Reference Behaviors of Improper Settings on MDCFG Table

Here is a non-exhaustive list of behaviors for handling improper settings on MDCFG Table:

1. correct the values to make the table have proper settings,
2. reject write accesses that produce improper settings, or
3. leave the improper setting in the MDCFG Table, but association of entries to an MD m that has an improper setting becomes:
   a. no entries belong to MD *m*, or
   b. no entries belong to any MDs from *m* to (`HWCFG0.md_num` - 1).

A.6. MDCFG Table Reduction

In the baseline specification, a complete IOPMP check involves a sequential, two-stage lookup: first the SRCMD Table (to find MD indexes) and then the MDCFG Table (to find the corresponding entry indexes). This data dependency on the MDCFG Table lookup can introduce a latency.

This application note provides a guideline for an optimized implementation model that **eliminates this second lookup stage**. This optimization reduces check latency, simplifies hardware logic, and saves area by

removing the need for the MDCFG Table storage.

A.6.1. Each Memory Domain Has Exactly *k* Entries

This format is the foundational mechanism for achieving the MDCFG Table optimization, and it is the basis for the Section A.8.2 and Section A.8.5. The primary motivation is to **remove the MDCFG Table and its associated lookup latency**.

This is achieved by defining that each memory domain contains exactly k entries, where k is a fixed value. By making k a constant, the range of entry indexes for a given MD m is no longer stored in a table but is **directly calculable** (e.g., Entry Indexes = `[m * k, (m * k) + k - 1]`).

This optimization breaks the sequential data dependency of the two-stage lookup, enabling a faster, simpler, and lower-area implementation. Implementations adopting this model can omit the MDCFG Table and the MDCFGLCK register entirely.

To indicate the k value, an implementation can have `md_entry_num` in `HWCFG3` (Hardware Configuration 3). `md_entry_num + 1` indicates the k value of the IOPMP instance.

k * `HWCFG0.md_num` *can be greater than* `HWCFG1.num_entry`. *Behavior of any entry with index* ≥ `HWCFG1.num_entry` *is described on IOPMP Entry and IOPMP Entry Array.*

For a particular case of k=1 (`HWCFG3.md_entry_num`=0), every memory domain has exactly one entry. Software can skip the concept of the memory domain when using such an IOPMP.

The k value also can be programmable. An implementation can use `HWCFG0.enable` as the lock bit of `HWCFG3.md_entry_num`.

To indicate the reduced table, an implementation can have `mdcfg_fmt` in `HWCFG3` (Hardware Configuration 3) to indicate what reduced table format is implemented in an IOPMP.

A.7. Run Out Memory Domains

In IOPMP specification, the support is capped at 63 memory domains. However, this section provides customization recommendations for situations that necessitate a larger number of memory domains.

A.7.1. Parallel IOPMP

Placing multiple IOPMP instances in parallel is a strategy to address two distinct system-level goals: (1) to scale beyond the 63-Memory Domain limit, and (2) to increase the aggregate **transaction checking throughput**.

By partitioning the system, multiple IOPMPs can operate concurrently. This arrangement is highly beneficial for complex SoCs that must handle a large number of simultaneous transactions. A routing mechanism must be placed in front of the parallel IOPMPs to direct an incoming transaction to its designated IOPMP for checking. This routing can be based on the transaction's address (address-based routing) or its RRID (RRID-based routing).

A.7.2. Cascading IOPMP

The cascading model is a powerful architecture for enabling **hierarchical security** in large, modular systems. It is particularly well-suited for integrating multiple, independently developed SoCs or subsystems, where each subsystem manages its own internal security policies.

In this policy, a transaction may traverse more than one IOPMP. An IOPMP at the boundary of a subsystem, referred to as an "IOPMP gateway," performs the initial check. Upon a successful check, it tags the outgoing transaction with a new, subsystem-level RRID.

This new RRID effectively represents that the transaction has been checked by its originating subsystem. Subsequent, outer-level IOPMPs can then manage permissions from a higher, more abstract "subsystem perspective" instead of needing to manage rules for every individual requester within that subsystem. This approach hides internal subsystem details, which is good for protecting intellectual property, and makes the overall system design more abstract, reusable, and modular.

To support this "IOPMP gateway" functionality, an implementation can have field `rrid_transl_en`, `rrid_transl`, and `rrid_transl_prog` in register `HWCFG3` (Hardware Configuration 3). `HWCFG3.rrid_transl_en` should be set to 1, and `HWCFG3.rrid_transl` is used to store the RRID to be tagged. `HWCFG3.rrid_transl_prog` indicates whether `HWCFG3.rrid_transl` is programmable or not. To lock `HWCFG3.rrid_transl`, write 1 to `HWCFG3.rrid_transl_prog`, which clears `HWCFG3.rrid_transl_prog` and is sticky to 0.

A.8. IOPMP Implementation Models

The IOPMP specification supports multiple implementation models that combine different SRCMD Table and MDCFG Table formats to address specific system requirements. These models provide convenient reference configurations for common use cases, enabling designers to select appropriate trade-offs between hardware complexity, flexibility, and performance.

A.8.1. Full Model

Configuration: **srcmd_fmt**=0, **mdcfg_fmt**=0

The Full Model is the default IOPMP configuration with complete SRCMD and MDCFG Table implementations, providing maximum flexibility and supporting all IOPMP extensions. This serves as the baseline for comparison with other optimized models.

A.8.2. Rapid-*k* Model

Configuration: **srcmd_fmt**=0, **mdcfg_fmt**=1, where k = (**md_entry_num** + 1)

The Rapid-k Model simplifies MDCFG Table lookup by implementing a fixed entry allocation scheme where each memory domain contains exactly k entries. This eliminates the need for MDCFG Table lookups, reducing access latency and hardware complexity while maintaining full SRCMD Table flexibility.

Characteristics:

- Full SRCMD Table with arbitrary RRID-to-MD associations
- Simplified MDCFG with constant k entries per MD
- Direct calculation: entry index = MD index × k + offset
- Reduced MDCFG lookup latency
- Supports SPS extension

Use cases: Systems with uniform memory region sizes, real-time systems requiring predictable access latency, applications with regular memory partitioning.

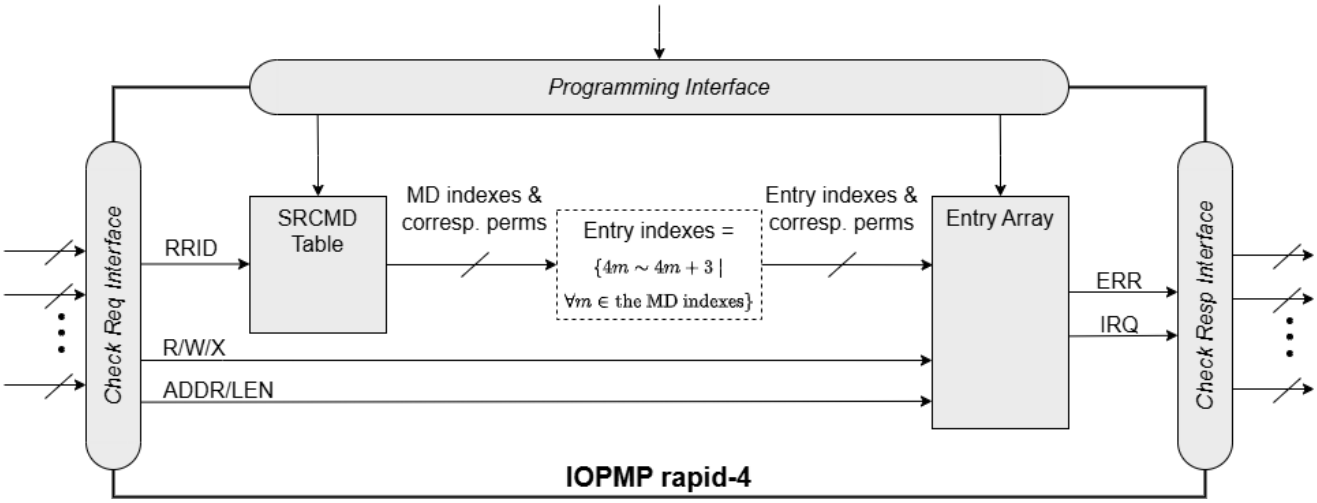

Figure 7. Block diagram of the Rapid-4 model implementation. The MDCFG Table is simplified to a constant mapping where each memory domain contains exactly four entries.

A.8.3. Dynamic-*k* Model

Configuration: **srcmd_fmt**=0, **mdcfg_fmt**=2, where k = (**md_entry_num** + 1)

The Dynamic-k Model provides runtime-programmable entry allocation similar to the Full Model but with a different MDCFG Table format. This format enables efficient dynamic reconfiguration of memory domain boundaries while maintaining full SRCMD Table capabilities.

Characteristics:

- Full SRCMD Table with arbitrary RRID-to-MD associations
- Alternative MDCFG Table format for dynamic configuration
- Runtime-programmable memory domain boundaries
- Supports SPS extension
- Optimized for dynamic memory management

Use cases: Systems requiring frequent memory domain reconfiguration, dynamic resource allocation scenarios, adaptive protection schemes.

A.8.4. Isolation Model

Configuration: **srcmd_fmt**=1, **mdcfg_fmt**=0

The Isolation Model implements direct RRID-to-MD mapping where RRID i exclusively associates with memory domain i. This provides strong isolation between requesters while maintaining flexible entry allocation through the full MDCFG Table. The model eliminates SRCMD Table hardware entirely, achieving significant area savings.

Characteristics:

- Direct RRID-to-MD mapping (RRID i → MD i)
- No physical SRCMD Table implementation
- Full MDCFG Table with flexible entry allocation
- Supports up to 63 RRIDs

- Strong requester isolation
- SPS extension not supported

Use cases: Embedded systems with isolated functional units, systems prioritizing strong isolation over shared resources, source-enforcement scenarios.

A.8.5. Compact-*k* Model

Configuration: **srcmd_fmt**=1, **mdcfg_fmt**=1, where k = (**md_entry_num** + 1)

The Compact-k Model maximizes area efficiency by combining both SRCMD and MDCFG simplifications. It implements direct RRID-to-MD mapping with fixed k entries per memory domain, eliminating both table lookup mechanisms. This represents the most area-optimized IOPMP configuration.

Characteristics:

- Direct RRID-to-MD mapping (RRID $i \rightarrow$ MD i)
- Fixed k entries per MD with constant allocation
- No SRCMD Table implementation
- Simplified MDCFG with constant entry allocation
- Minimum hardware resource utilization
- Direct calculation: entry index = RRID × k + offset
- SPS extension not supported

Use cases: Resource-constrained embedded systems, cost-sensitive applications, simple protection scenarios with isolated requesters and uniform memory regions.

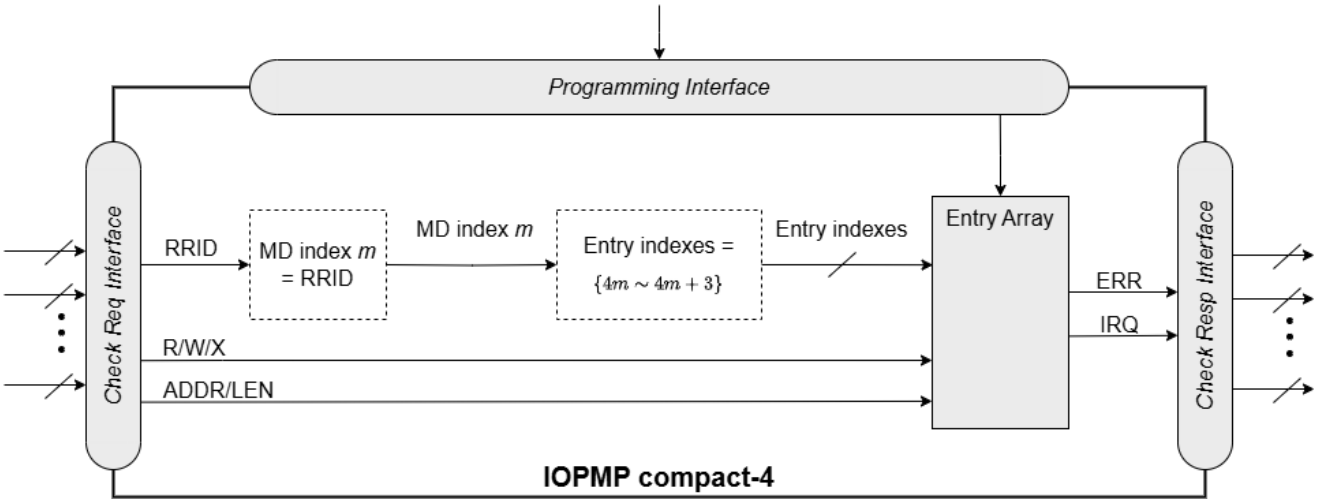

Figure 8. Block diagram of the Compact-4 model implementation, combining SRCMD Format 1 and MDCFG Format 1 for maximum area efficiency.

These models serve as implementation guidelines, allowing system designers to choose configurations that best match their area constraints, performance requirements, and protection granularity needs. The selection involves trade-offs between hardware complexity, flexibility, isolation strength, and configuration overhead.